

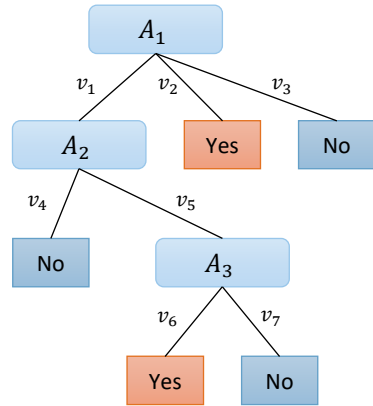
Model Selection

The model $\mathcal{M}$ of a machine learning system **encodes**, **stores** and **retrieves** the outcome of the learning process. The adaption of $\mathcal{M}$ by a learning algorithm $\mathcal{A}$ to find an optimal model $\mathcal{M}^*$ corresponds to a search in a **hypothesis space** $\mathcal{H}$:

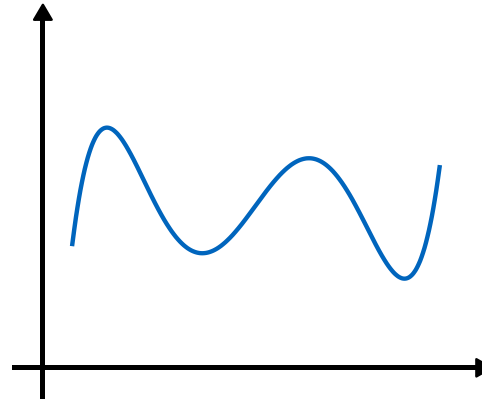
$$\mathcal{M}^* = \operatorname{argmin}_{\mathcal{M} \in \mathcal{H}} \mathcal{L}(\mathcal{M})$$

- In most practical applications, it is not possible to find a global minimum $\mathcal{M}^*$; most learning algorithms therefore search for **local optima**
- $\mathcal{H}$ determines which **aspects** of the data are captured (e.g. simple statistics or probability distributions) and how they are **represented** (e.g. implicit or human-readable)
- Different hypothesis spaces support the incorporation of **a priori knowledge** about the learning task to different degrees (e.g. neural networks or logic)

Examples of Common Hypothesis Spaces



Decision Trees

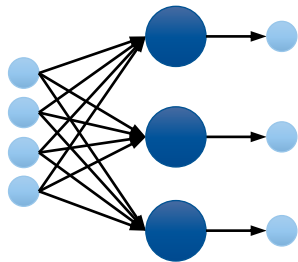


Polynomials

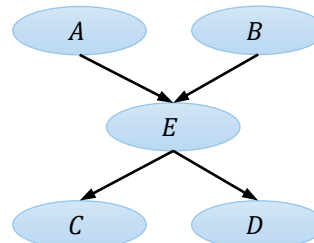
$$A \wedge B \Rightarrow C$$

$$\forall x: P(x)$$

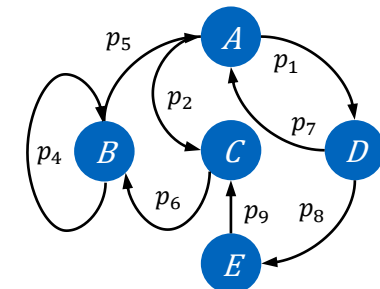
Logic



Neural Networks



Bayesian Networks



Markov Models

Ensemble Methods and Boosting

Combining Hypotheses to Ensembles

Ensemble methods in machine learning are a simple way to **extend hypothesis spaces** by combining a set of hypotheses $h_1, h_2, \dots, h_n \in \mathcal{H}$ to a new hypothesis $h^* \in \mathcal{H}^n$.

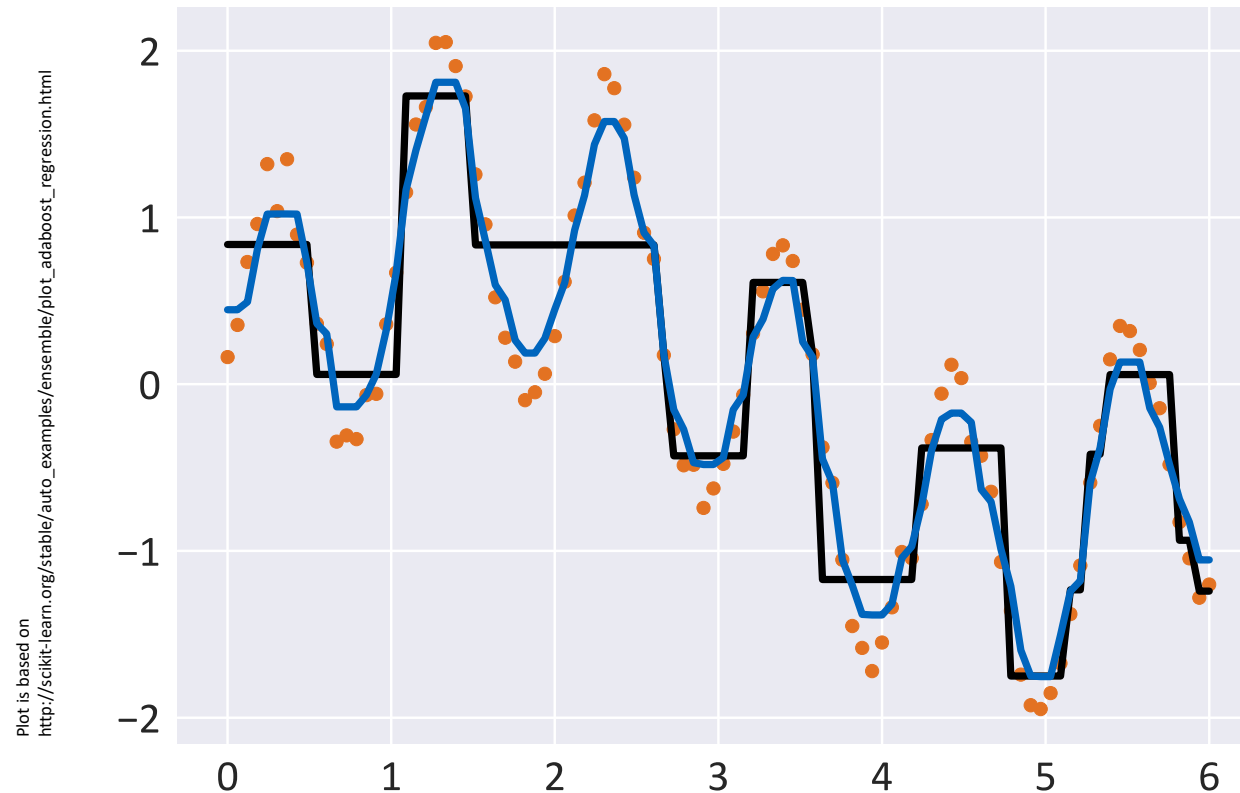
→ Is it possible to build a **strong learner** by combining several **weak learners** with accuracies that are just slightly above chance?

Boosting

Boosting algorithms compute a strong learner by **incrementally** constructing an ensemble of hypotheses:

- Every training sample $s_i \in \mathcal{S}$ is **assigned a weight** w_i ; initially, all weights are set to the same value
- Weights of incorrectly learned samples are **increased**
- The training of new hypotheses **focusses** on samples with **high weights**

Example: Boosted Decision Tree Regression



Decision tree regression with a single regression tree (**black**) and with 300 boosted decision trees (**blue**). Boosting was computed with the **AdaBoost** algorithm.

Underfitting, Overfitting, and Generalization

The **performance** of a hypothesis $h \in \mathcal{H}$ can be characterized by how well it **predicts seen data** from the training set and **unseen data**:

- **“Fit”**: the performance of h the training data (i.e. the value of the of the objective function $\mathcal{L}$ for h and S)
- **Generalization**: predictive performance of h on data that were not considered during the training phase

Depending on the expressiveness of the hypothesis space $\mathcal{H}$ and the properties of the objective function $\mathcal{L}$, two different issues can occur during learning:

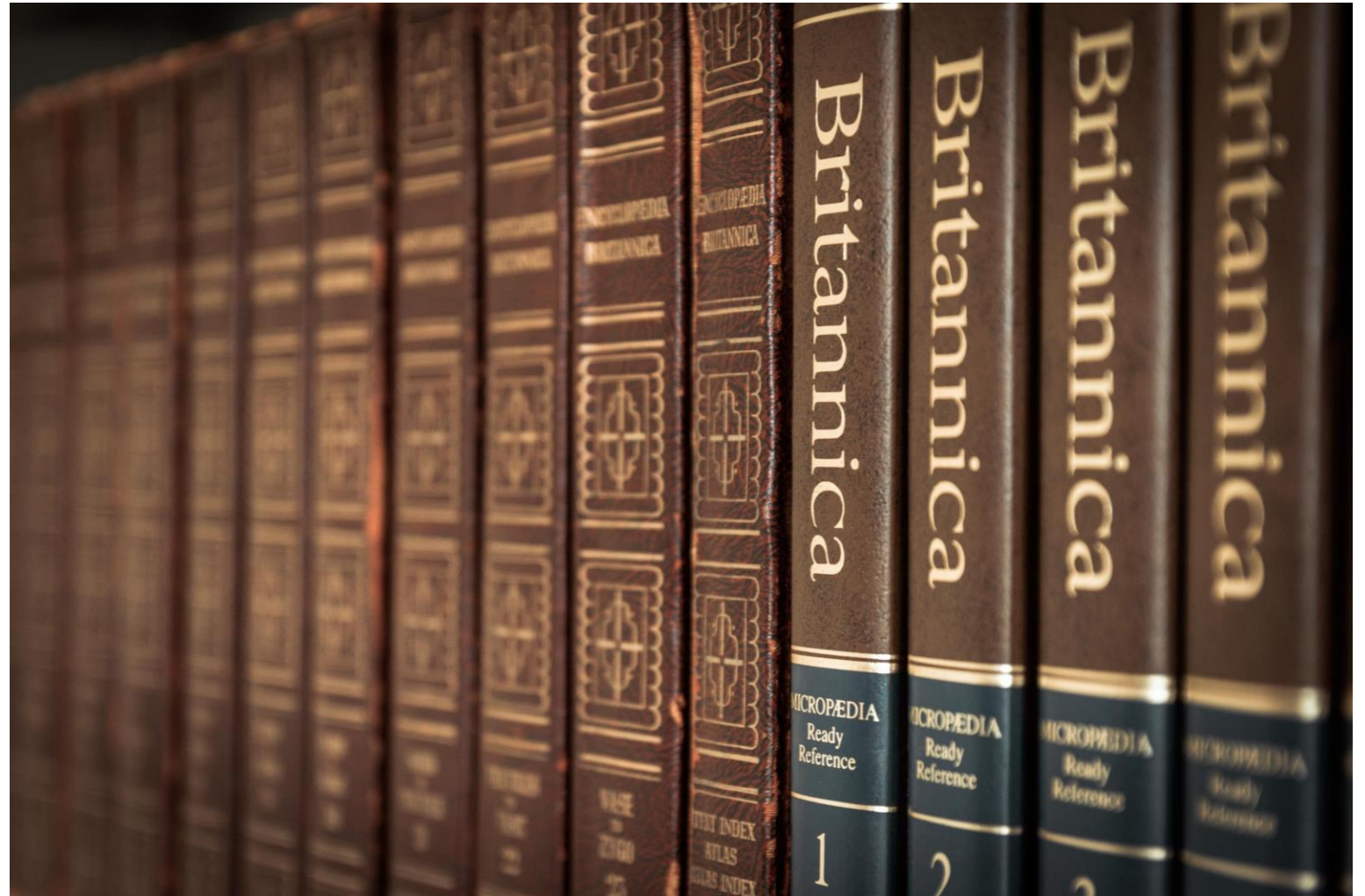
- **Underfitting**: h fits the training data poorly and does not model the underlying process because $\mathcal{H}$ is not expressive enough
- **Overfitting**: h fits the training data very well but does not model the underlying process because it does not generalize well

An Analogy to Cognition

An overfitted model “learns the data by heart” without **understanding** the underlying rules.

This is like a student who learns all exercises in a math book by heart without having understood the underlying theorems and formulas. She will only be able to answer questions that are identical to those in the book.

Adapted and extended from Wikipedia



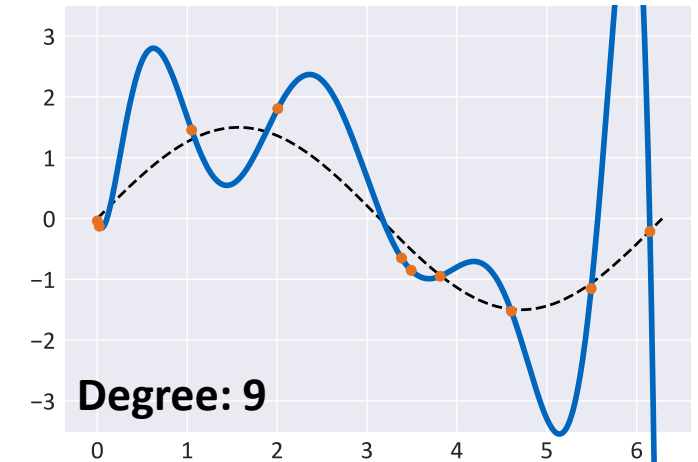
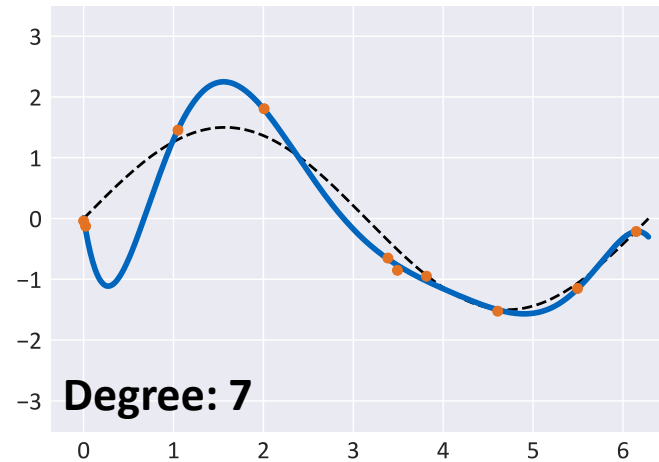
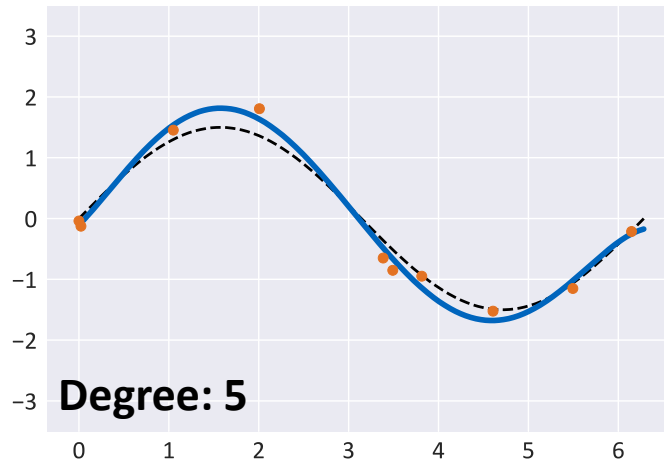
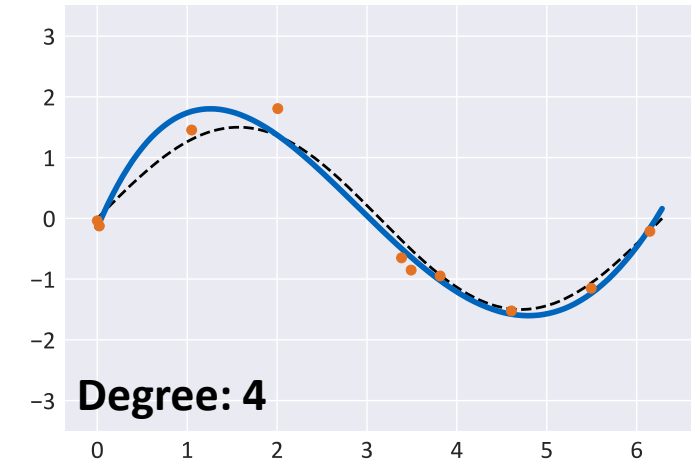
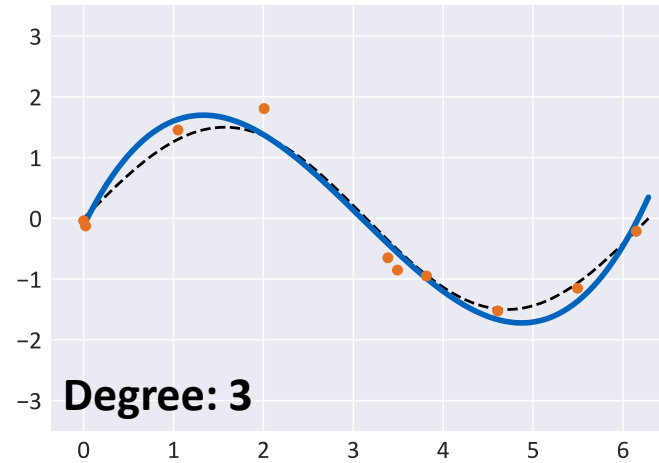
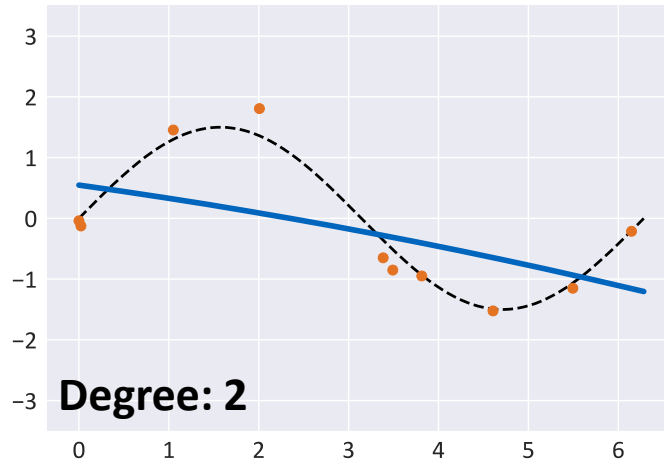
Occam's Razor

Models h that overfit a given dataset S have more parameters than required to capture the properties of the process that generates S . In general, it is desirable to choose a model that is **as simple as possible**. This **law of parsimony** is stated in the principle of Occam's razor :

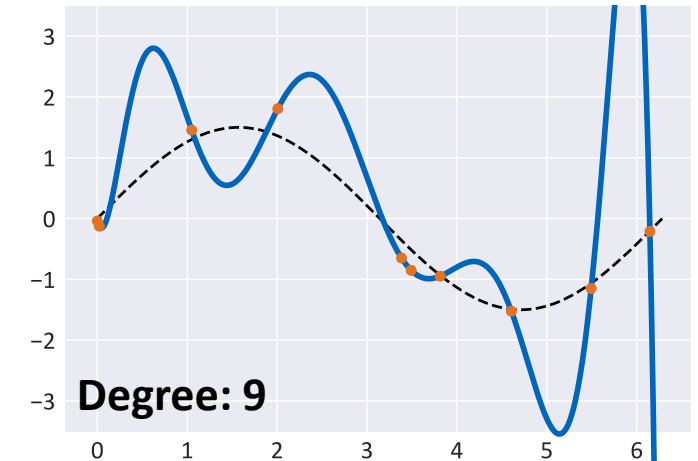
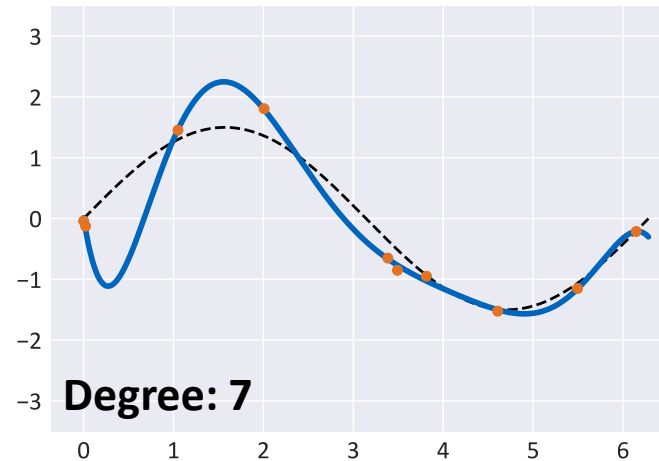
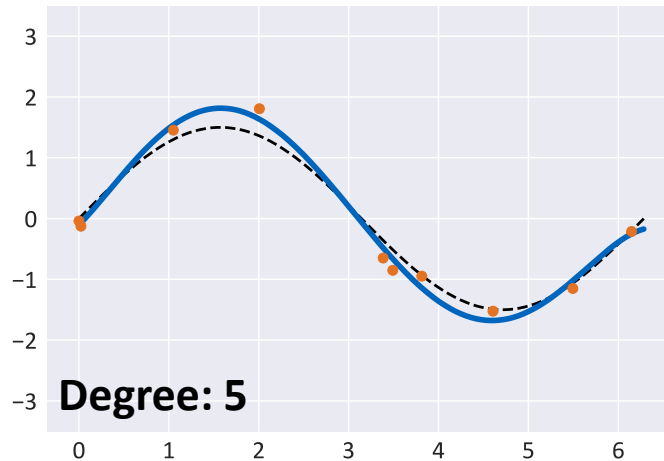
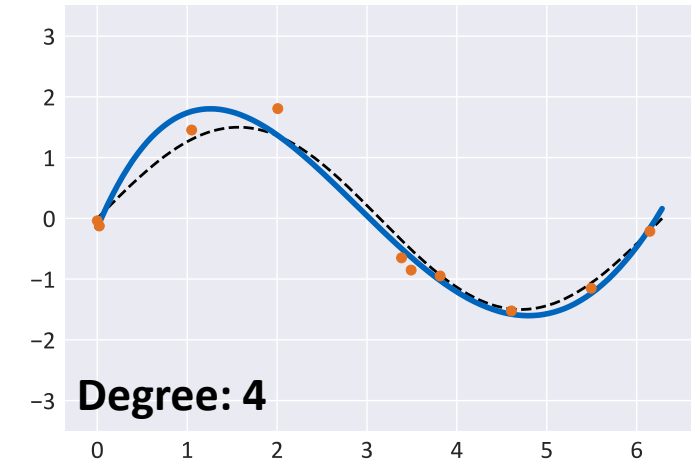
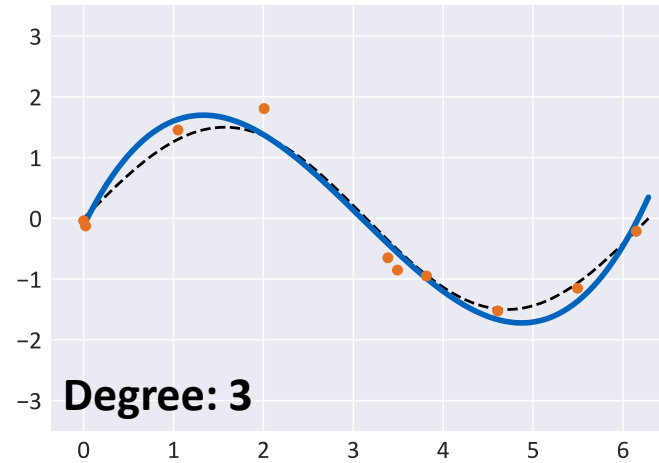
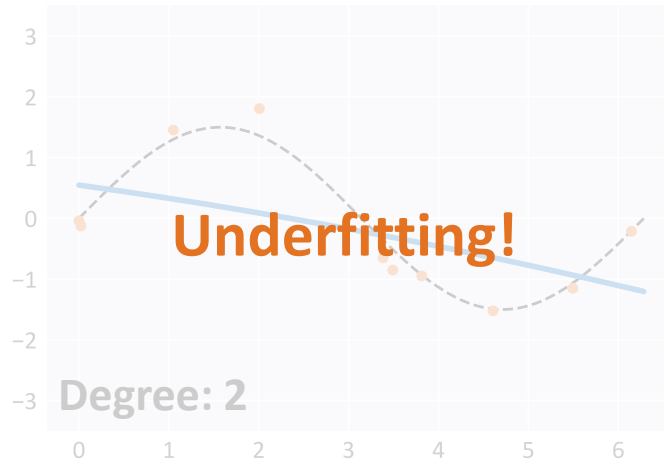
“Of two competing theories, the simpler explanation of an entity is to be preferred.”

Encyclopedia Britannica

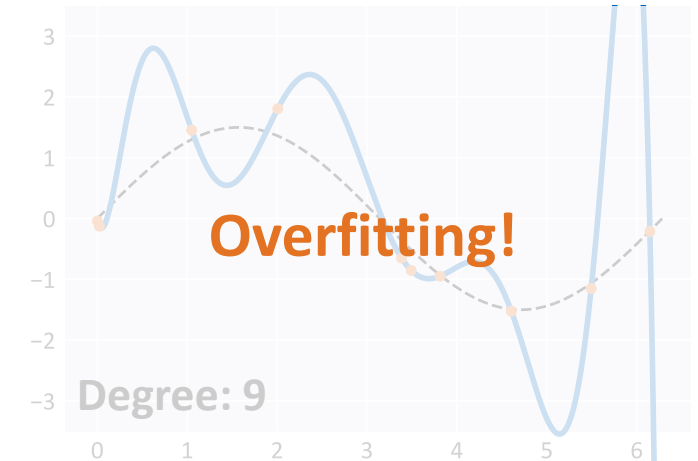
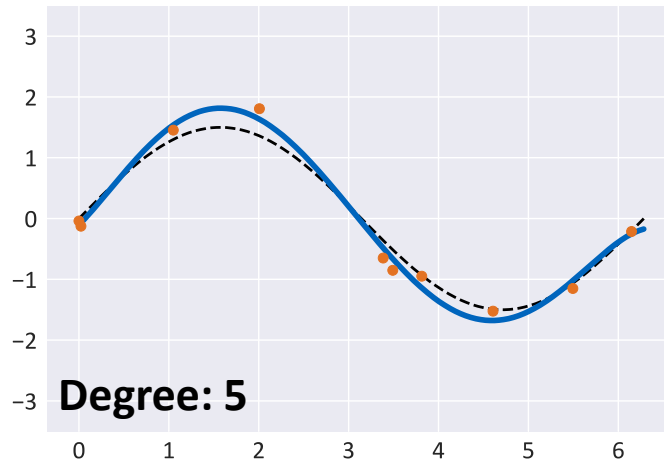
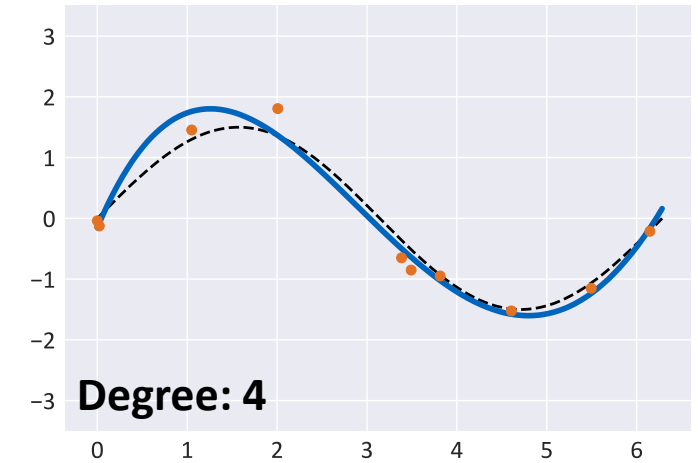
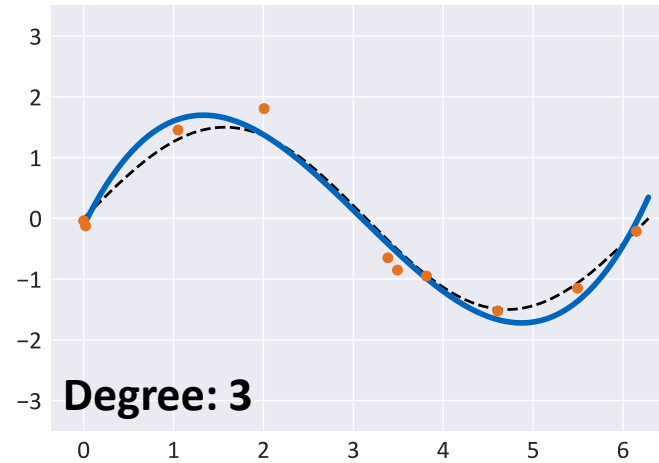
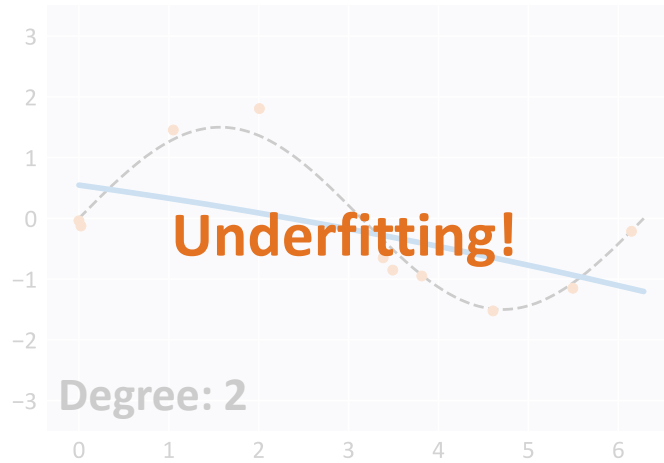
Example: Fitting Polynomials to a Dataset



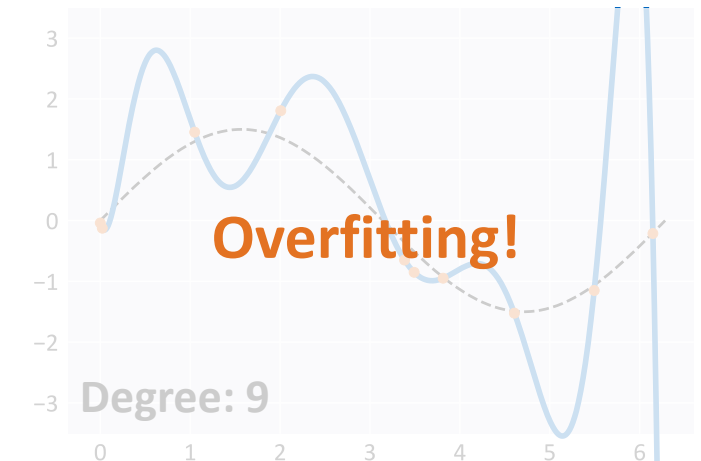
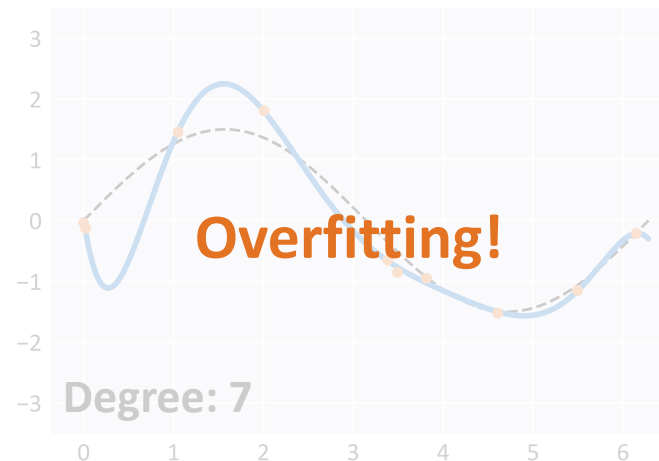
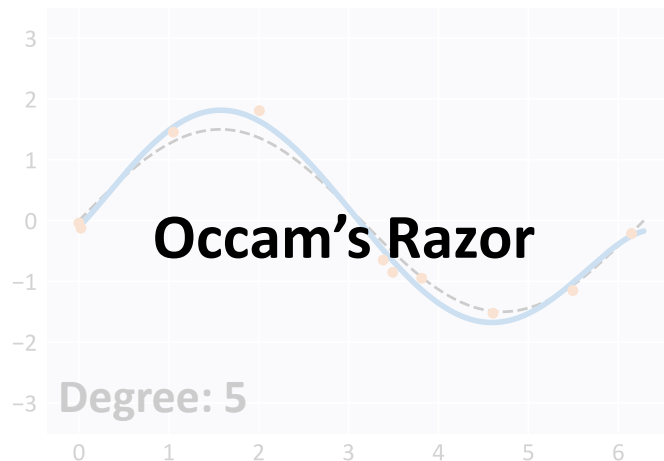
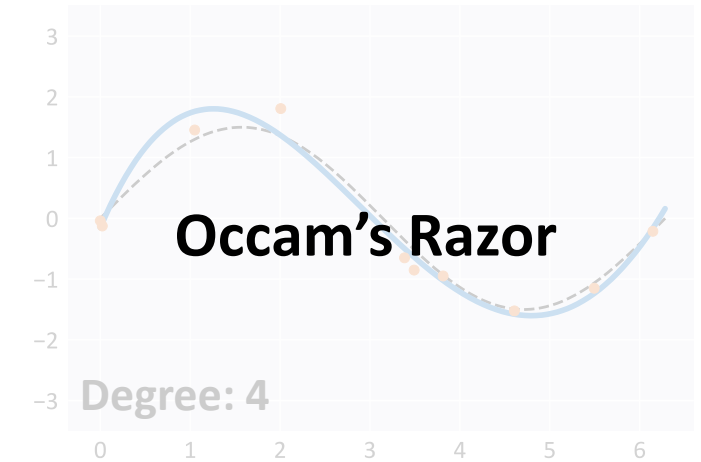
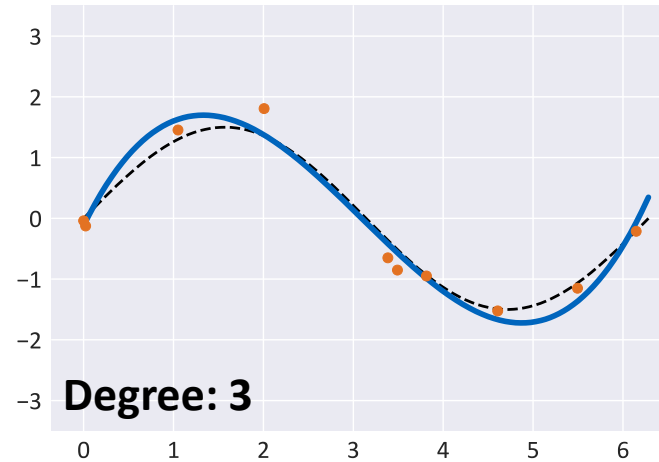
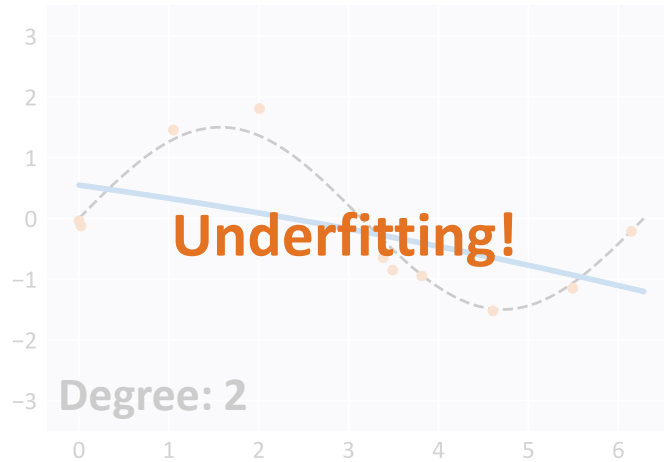
Example: Fitting Polynomials to a Dataset



Example: Fitting Polynomials to a Dataset



Example: Fitting Polynomials to a Dataset



Generative and Discriminative Models

Probabilistic models cannot only be trained to **discriminate** data (compute the prediction y for an input x) from the distribution $\mathcal{D}$ that generates $\mathcal{S}$ but also to model $\mathcal{D}$ directly. In general, samples (x, y) drawn from $\mathcal{D}$ are distributed according to $P(x, y)$:

- **Discriminative models** are based on the posterior probabilities $P(y|x)$
- **Generative models** are based on the prior probabilities $P(x|y)$; predictions can be computed by applying Bayes' theorem:

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

Generative models are **compact representations** of the training data that have considerably less parameters than the dataset $\mathcal{S}$.

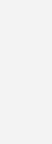
Demo: Generative Modeling of Faces

Online Demo

The online demo needs to load around 35 MB's worth of model parameters, which might take some time depending on your connection speed.



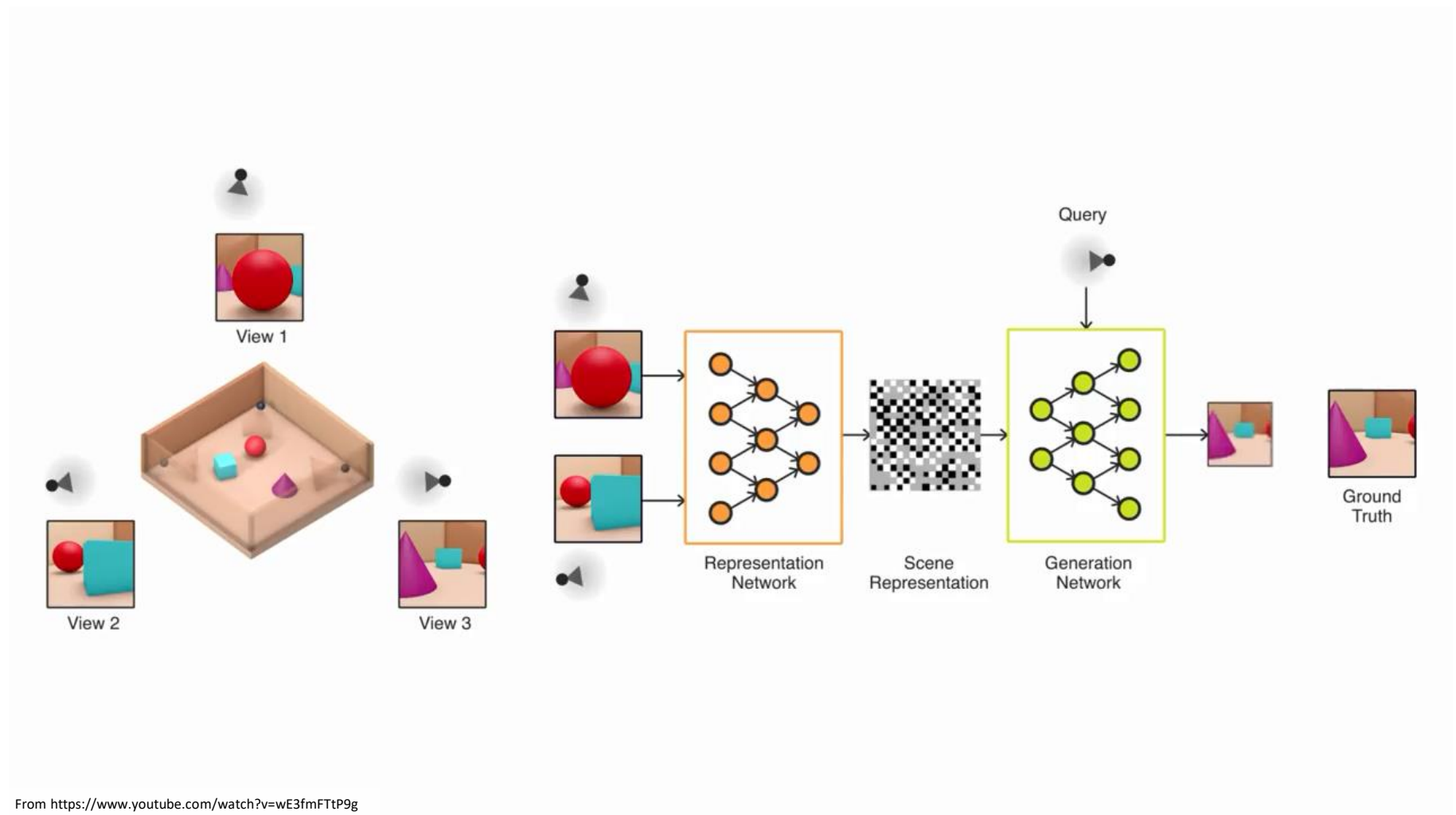
Random

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
														
15	16	17	18	19	20	21	22	23	24	25	26	27	28	
														

http://vdumoulin.github.io/morphing_faces

Generation of Camera Views in 3D Scenes

Generative Query Networks learn internal scene representations in a **representation network** and generate predict the visual appearance of the scene through a **generation network**.



Detection of Overfitting

Overfitting can be detected by applying the model h to **unseen data samples** and computing the objective function $\mathcal{L}$. Especially in supervised learning, the data are therefore split into **three different and disjoint subsets**:

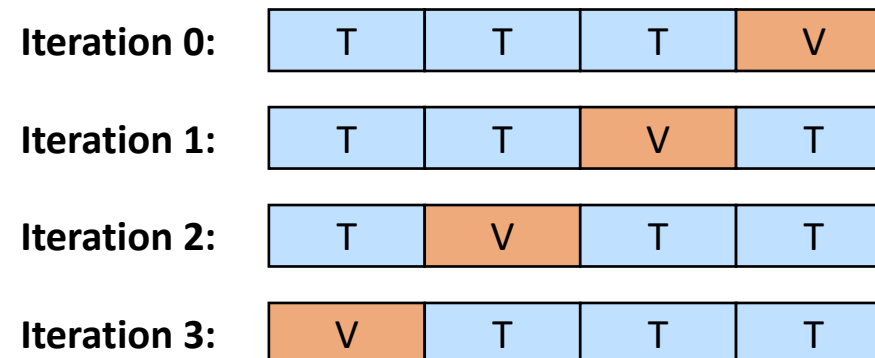
- **Training set**: the samples used in the training phase by the learning algorithm to search for a hypothesis h in the hypothesis space $\mathcal{H}$
- **Validation set**: a set of samples that are used to assess the performance of a hypothesis h that was computed in the training phase; based on the performance of h , the parameters of the training phase can be adjusted
- **Test set**: a set of samples (or real-world data) that is used to assess the performance of the final model

If h is modified based on the performance of the test set, the test set is invalidated!

Cross-Validation

In typical real-world applications, data is generated by a **physical processes** in the real world (e.g. a robot interacting with an environment) and therefore **limited**. **k-fold cross-validation** enables an efficient use of data at the cost of computational complexity:

- The dataset is partitioned into **k subsets** and learned in **k iterations**
- In every iteration, a different subset is selected as validation set
- The overall performance corresponds to the **averaged performances** of the k iterations



Avoidance of Overfitting

Regularization

Overfitting is only possible with hypotheses h that are **complex enough** to capture statistical features that do not explain the data (e.g. noise). A **regularization term** in the objective function $\mathcal{L}$ guides the learning process towards simpler solutions by **punishing complexity**.

More Training Data

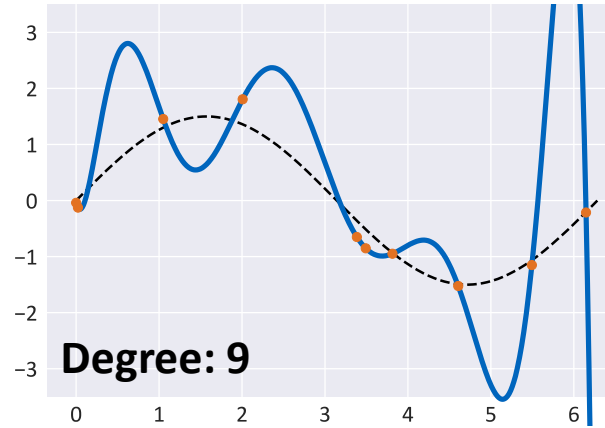
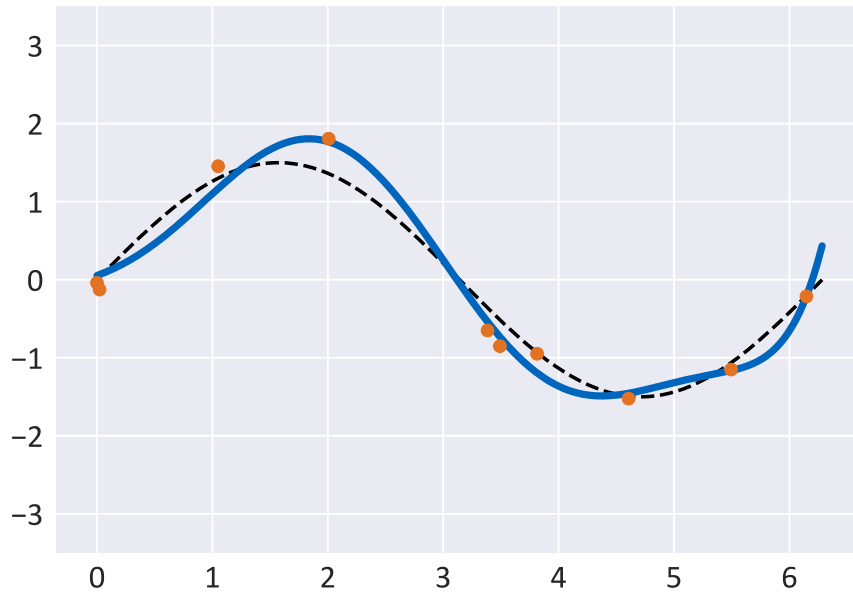
Overfitting can be reduced by **increasing the size** (i.e. the complexity) of the dataset.

Dataset Augmentation

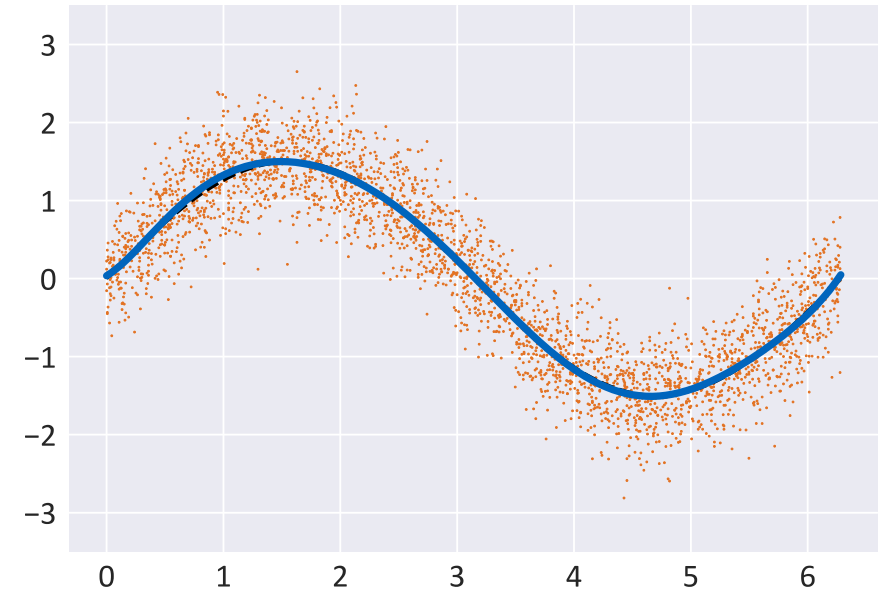
If not enough training data is available, the size of the dataset can be increased by **applying transformations** to the training samples (i.e. add noise, apply shifts, translations, and rotations, etc.).

Example: Fitting Polynomials

Regularization



More Data



Classification Criteria of Learning Methods

Type of Learning

- Supervised learning
- Semi-supervised learning
- Reinforcement learning
- Unsupervised learning

Model

- Deterministic – Stochastic
- Parametric – Nonparametric
- Generative – Discriminative

Value Domain

- Discrete (classification)
- Continuous (regression)

Reasoning

- Inductive
- Deductive
- Transductive

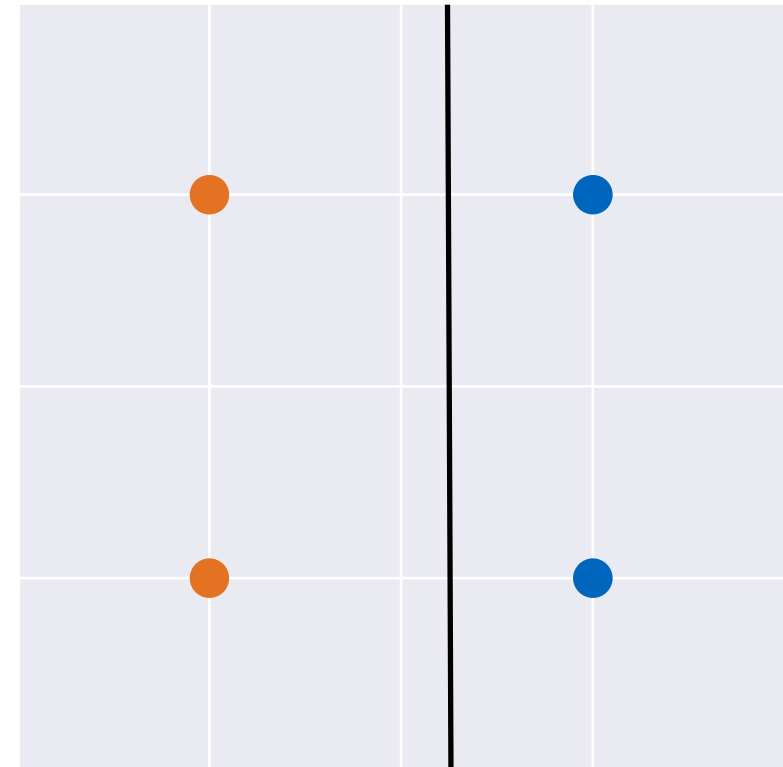
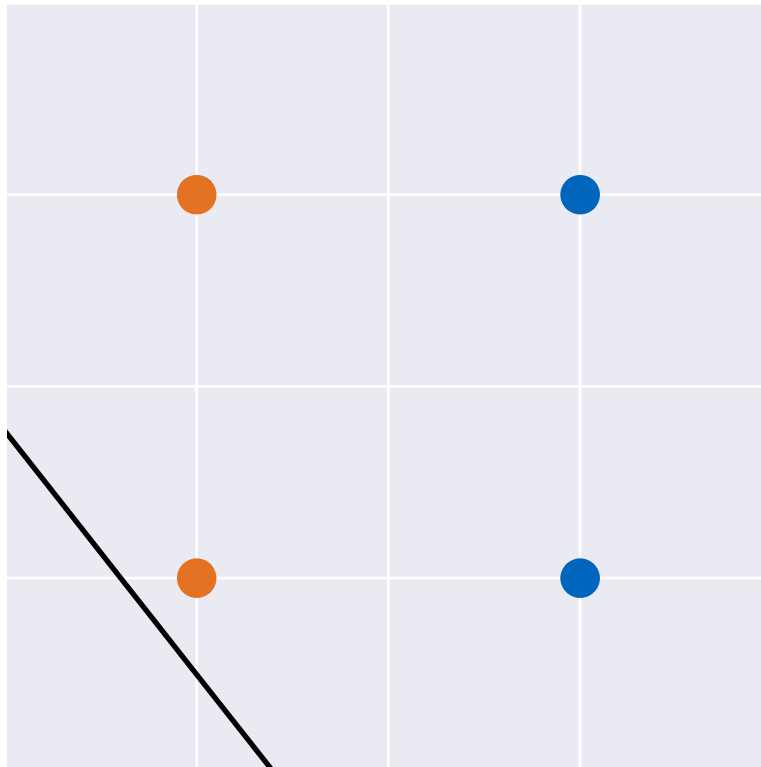
Online Learning vs. Offline Learning

Support for A Priori Knowledge

“Passive” Learning vs. “Active Learning”

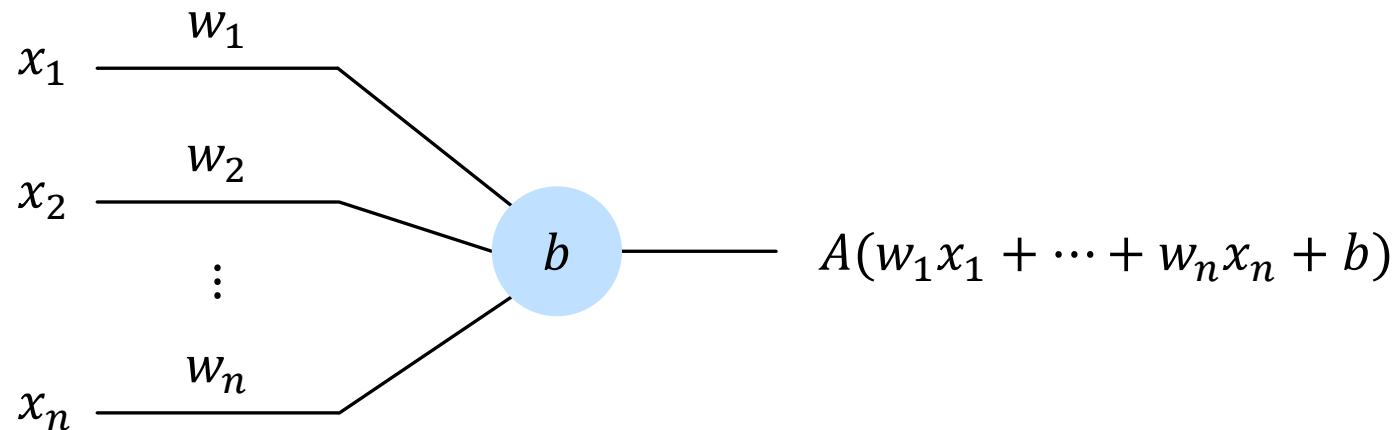
A First Example: The Perceptron Learning Rule

Goal: Separate Points through a Line



Artificial Neurons

An **analog artificial neuron** is defined by its **synaptic weights** and an **activation function**:



The output $n(x)$ of a neuron n with activation function A , synaptic weights w , bias b , and input x is computed as follows:

$$n(x) = A(w_1x_1 + \dots + w_nx_n + b) = A(w^T x + b)$$

The Perceptron

The perceptron is a **linear classifier** that is based on a **single neuron** with a digital threshold function:

$$y(x) = f(w^T x) \text{ with } f(x) = \begin{cases} +1 & \text{if } x \geq 0 \\ -1 & \text{if } x < 0 \end{cases}$$

With $y_n \in \{-1, 1\}$, the following inequality holds for all correctly classified samples:

$$(w^T x_n) y_n > 0$$

The **perceptron criterion** punishes only incorrectly classified samples $n \in \mathcal{M}$:

$$E_P(w) = -\sum_{n \in \mathcal{M}} (w^T x_n) y_n$$

The Perceptron Learning Rule

Computing the gradient of $E_P(w)$ yields the following expression:

$$\nabla E_P(w) = -\sum_{n \in \mathcal{M}} \nabla(w^T x_n) y_n = -\sum_{n \in \mathcal{M}} x_n y_n$$

The online weight update rule can be derived by applying **stochastic gradient descent** and adding learning rate η :

$$\Delta w = \eta x_n y_n$$

Finally, the perceptron learning rule can be derived by applying the weight update:

$$w^{t+1} = w^t + \Delta w = w^t + \eta x_n y_n \text{ for all } n \in \mathcal{M}$$

$$w^{t+1} = w^t + \eta'(y_n - f^t(x_n))x_n \text{ for all } n$$

Convergence of the Perceptron Learning Rule

A **single update step** of the perceptron learning algorithm **reduces** the error of the **corresponding** misclassified sample:

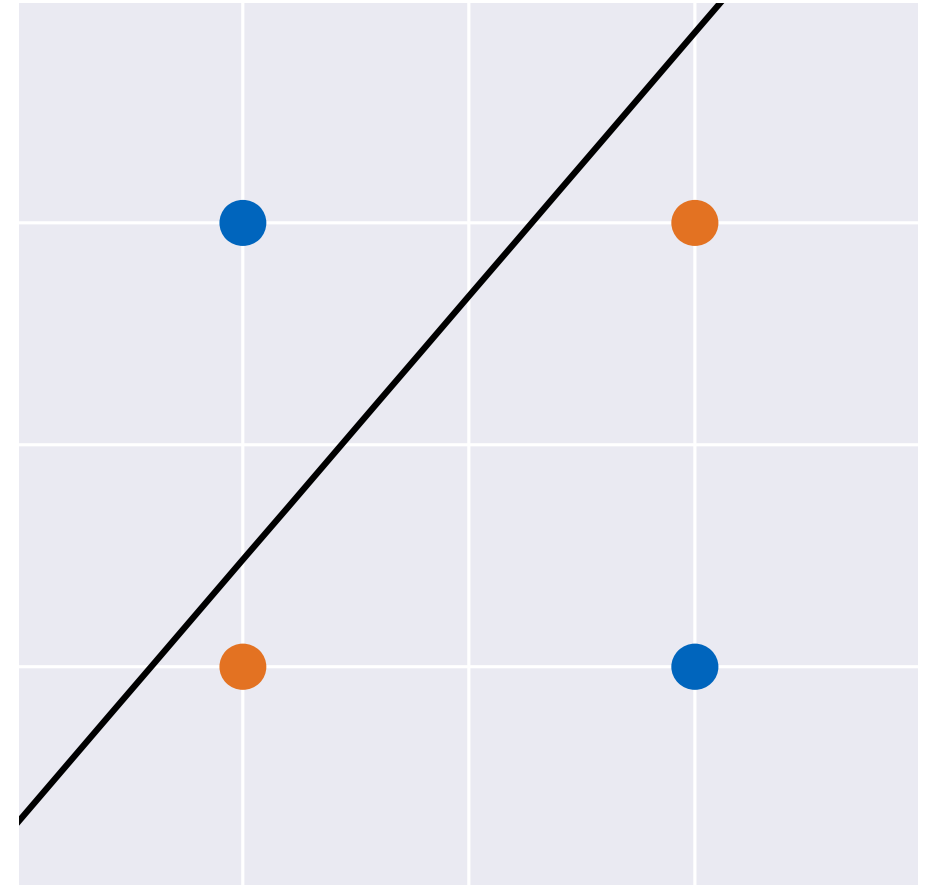
$$-w^{t+1T} x_n y_n = -w^{tT} x_n y_n - \eta (x_n y_n)^T (x_n y_n) < -w^{tT} x_n y_n$$

In general, it can be proven that the perceptron learning rule has the following properties:

- If a solution exists, i.e. if the data set is linearly separable, then the perceptron learning algorithm finds a solution within a **finite** number of steps (**perceptron convergence theorem**)
- The solution computed by the algorithm depends on the **initialization of the parameters** and the **order of presentation** of the training samples
- The algorithm does **not converge** for **not linearly separable** data sets

Task 1: The XOR Problem

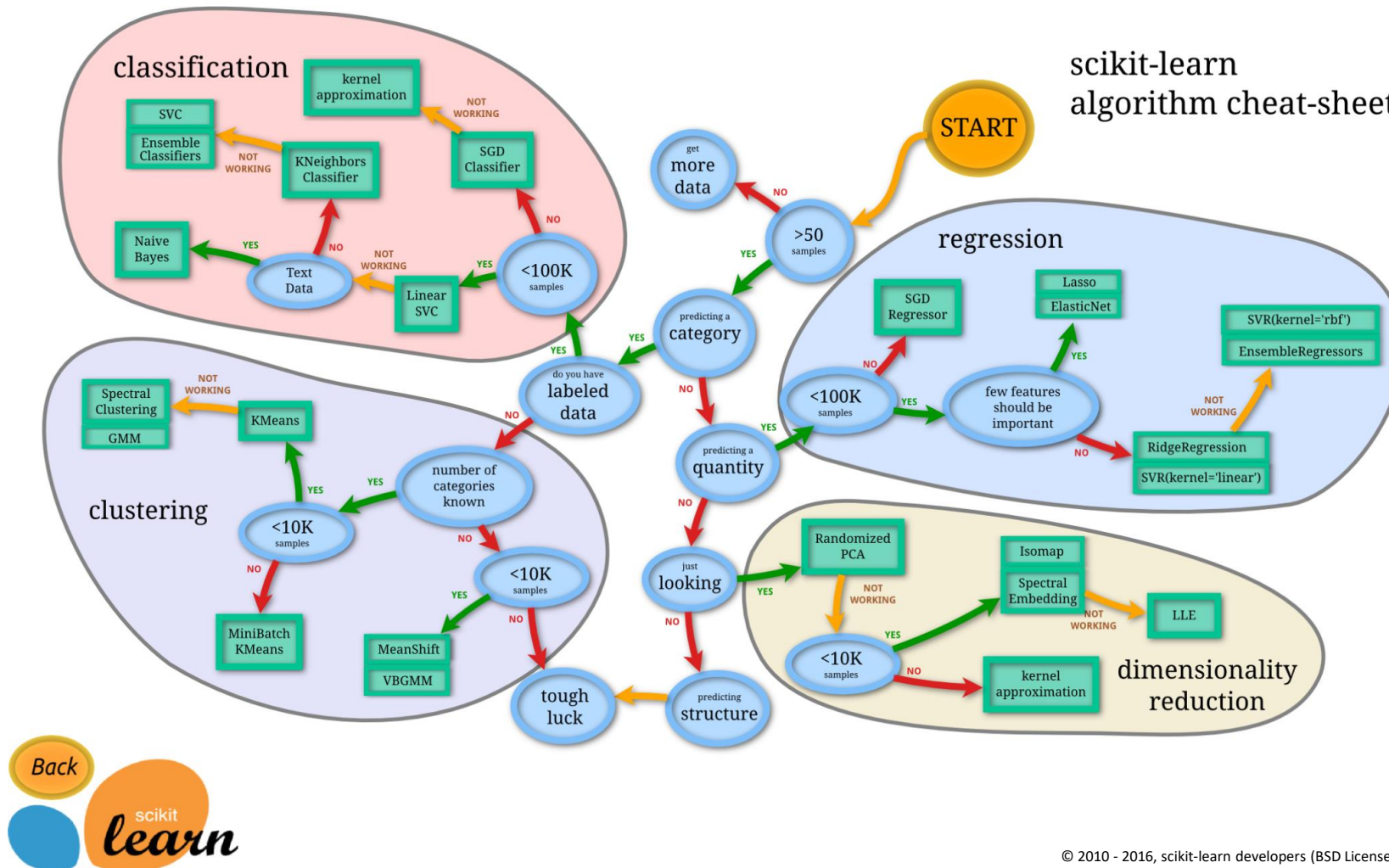
- The Perceptron learning rule only converges for linearly **separable data sets**
- It therefore cannot classify the **XOR dataset** correctly
- This finding led to an AI winter and the widespread **abandonment of connectionism** for almost two decades



Supervised Learning

Basic Regression and Classification

Machine Learning with scikit-learn



© 2010 - 2016, scikit-learn developers (BSD License)

Regression: Task Definition

Input

- A dataset $\mathcal{S} = \{(x_0, y_0), (x_1, y_1), \dots, (x_N, y_N)\}$ with $x_i \in \mathbb{X}^K$ and $y_i \in \mathbb{Y}^L$
- $\mathbb{Y}^L$ is **continuous** space (e.g. $\mathbb{R}^L$)
- A **loss function** $\mathcal{L}: \mathbb{Y}^L \times \mathbb{Y}^L \rightarrow \mathbb{R}$

Desired Output

A regressor $h: \mathbb{X}^K \rightarrow \mathbb{Y}^L$ that predicts y_n for a given x_n with minimum loss $\mathcal{L}$.

Linear Basis Function Models

Linear basis function models are a class of hypothesis spaces that are **linear combinations** of functions on the samples x_n :

$$h_w(x_n) = w_0 + \sum_{j=1}^M w_j \phi_j(x_n)$$

- The functions $\phi_j(\cdot)$ are the **basis functions** of the model
- When setting $\phi_0(\cdot) = 1$ we can write $y(x_n, w) = \sum_{j=0}^M w_j \phi_j(x_n) = w^T x_i$
- The values $\phi_j(x_n)$ are called **features** of x_n

Examples of Basis Functions

Linear Functions

$$\phi_j(x_n) = x_n \text{ (linear regression)}$$

Powers of x_i

$$\phi_j(x_n) = x_n^j \text{ (polynomial regression for } x_i \in \mathbb{R})$$

Radial Basis Functions

$$\phi_j(x_n) = \exp\left(-\frac{(x_n - \mu_j)^2}{2s^2}\right)$$

Sigmoid Functions

$$\phi_j(x_n) = \sigma\left\{\frac{x_n - \mu_j}{s}\right\} \text{ with } \sigma(z) = \frac{1}{1 + \exp(-z)}$$

Computing the Error for a Weight Vector w

The Sum-of-Squares Error Function

The sum-of-squares error of a hypothesis h_w with weights w_j on a dataset $\mathcal{S}$ is defined as

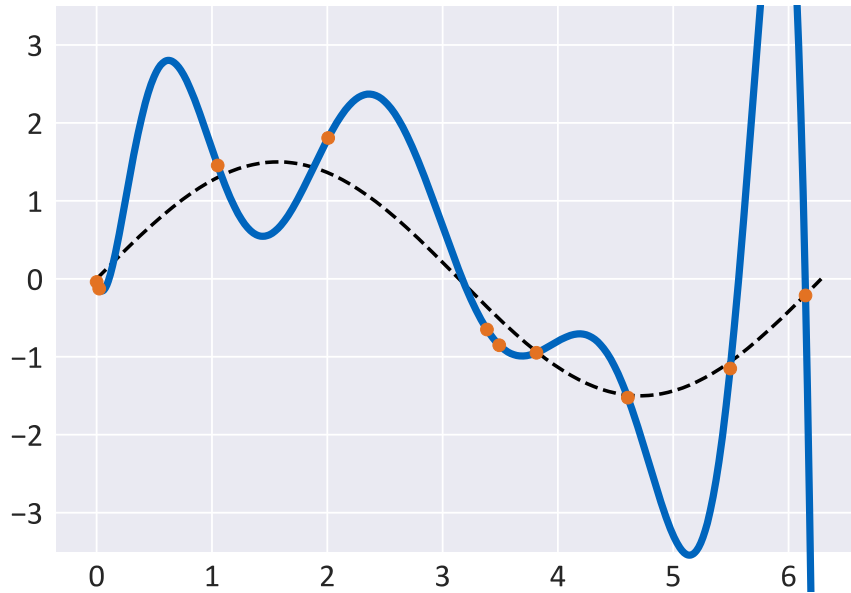
$$\mathcal{L}_{\mathcal{S}}(h_w) = \frac{1}{2} \sum_{n=1}^N (h_w(x_n) - y_n)^2$$

Optionally, it is possible to add a **regularization term** that punishes complex solutions:

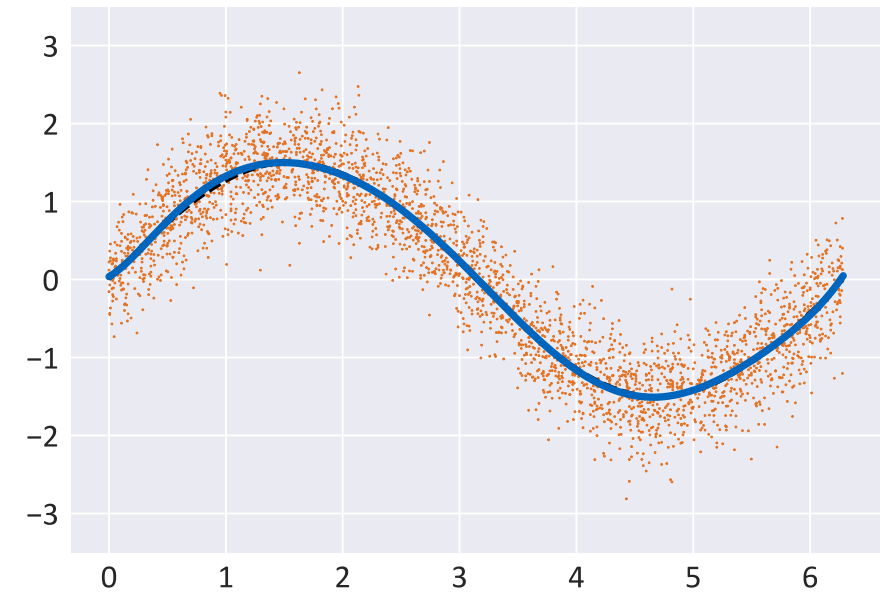
$$\mathcal{L}_{\mathcal{S}}^*(h_w) = \frac{1}{2} \sum_{n=1}^N (h_w(x_n) - y_n)^2 + \frac{\lambda}{2} \|w\|^2$$

→ Learning a locally optimal weight vector w corresponds to **minimizing** $\mathcal{L}$ or $\mathcal{L}^*$ with respect to w

Interpolation vs. Regression



The interpolation function $f(\cdot)$ must be **consistent** with $\mathcal{S}$: $f(x_i) = y_i$.



The hypothesis $h(\cdot)$ should **minimize** $\mathcal{L}$ and **generalize** well to new samples.

Learning w with Least Squares

Input

- A dataset $\mathcal{S} = \{(x_0, y_0), (x_1, y_1), \dots, (x_N, y_N)\}$
- A hypothesis space $\mathcal{H} = \{w_0\phi_0(x) + \dots + w_M\phi_M(x) | w_i \in \mathbb{R}\}$

Goal

Minimize

$$\mathcal{L}_{\mathcal{S}}(h_w) = \frac{1}{2} \sum_{n=1}^N (h_w(x_n) - y_n)^2$$

→ Find a set of weights w^* with $\nabla \mathcal{L}_{\mathcal{S}}(h_w) = 0$:

$$\frac{\partial}{\partial w_l} \mathcal{L}_{\mathcal{S}}(h_w) = 0 \text{ for } w_0, w_1, \dots, w_M$$

Learning w with Linear Least Squares

The Least Squares Solution

A hypothesis h_{w^*} that minimizes $\mathcal{L}_{\mathcal{S}}(\cdot)$ can be computed by solving the equation below for w :

$$\phi^T \phi w^* = \phi^T y \rightarrow w^* = (\phi^T \phi)^{-1} \phi^T y$$

In the above equation, ϕ and y are defined as follows:

$$\phi = \begin{pmatrix} \phi_0(x_0) & \cdots & \phi_M(x_0) \\ \vdots & \ddots & \vdots \\ \phi_0(x_N) & \cdots & \phi_M(x_N) \end{pmatrix} \quad y = \begin{pmatrix} y_0 \\ \vdots \\ y_N \end{pmatrix}$$

A Geometrical Interpretation

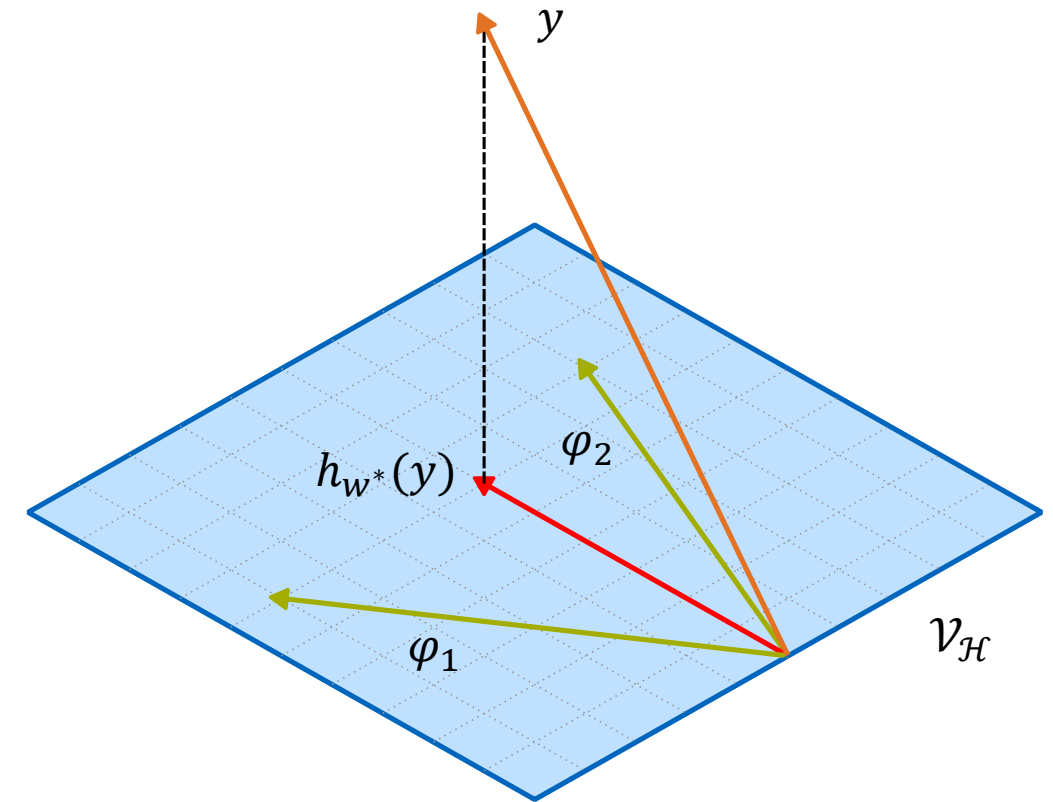
The minimization with least squares can be interpreted as a **orthogonal projection** of the vector

$$y = (y_0, y_1, \dots, y_n) \in \mathbb{R}^{N+1}$$

to the vector

$$h_w^*(y) = (h_w^*(x_0), \dots, h_w^*(x_n))$$

in the subspace $\mathcal{V}_{\mathcal{H}} \in \mathbb{R}^{M+1}$ that is spanned by the column vectors φ_i of the matrix ϕ (if M is smaller than N).



Learning w with Maximum Likelihood

Probabilistic Modeling

The prediction error of a hypothesis h_w can be interpreted as a noise term ϵ :

$$y_n = h_w(x_n) + \epsilon$$

Usually, the noise is modeled with a **Gaussian distribution** with **mean** $h_w(x_n)$ and **precision** (inverse variance) β :

$$P(y_n|x_n, w, \beta^{-1}) = \mathcal{N}(y_n|h_w(x_n), \beta^{-1})$$

Computing the optimal prediction for a given set of parameters is then equal to computing the expectation value $E[y_n|x_n]$:

$$E[y_n|x_n] = \int y_n P(y_n|x_n) dy_n = h_w(x_n)$$

Learning w with Maximum Likelihood

Maximum Likelihood

For a given dataset $\mathcal{S}$, the weights w should be computed so that they **maximize the likelihood** of the target values y_n for the input x_n . This amounts to maximizing the following probability:

$$\begin{aligned} P(y|x, w, \beta) &= \prod_{n=1}^N \mathcal{N}(y_n | h_w(x_n), \beta^{-1}) \\ &= \prod_{n=1}^N (2\pi\beta^{-1})^{-\frac{1}{2}} \cdot \exp\left(-\frac{\beta}{2} (y_n - h_w(x_n))^2\right) \end{aligned}$$

Maximizing the log-Likelihood

Instead of maximizing $P(y|x, w, \beta)$, it is often easier to maximize $\ln P(y|x, w, \beta)$:

$$\ln P(y|x, w, \beta) = -\frac{\beta}{2} \sum_{n=1}^N (h_w(x_n) - y_n)^2 + \frac{N}{2} \ln \beta - \frac{N}{2} \ln 2\pi$$

Learning w with Maximum Likelihood

Maximizing the log-Likelihood

Instead of maximizing $P(y|x, w, \beta)$, it is often easier to maximize $\ln P(y|x, w, \beta)$:

$$\ln P(y|x, w, \beta) = -\frac{\beta}{2} \sum_{n=1}^N (h_w(x_n) - y_n)^2 + \frac{N}{2} \ln \beta - \frac{N}{2} \ln 2\pi$$

→ Minimize $\sum_{n=1}^N (h_w(x_n) - y_n)^2 = 2 \cdot \mathcal{L}_{\mathcal{S}}(h_w)$

For linear basis function models, when assuming a Gaussian distribution of the prediction error, maximizing the log-likelihood is **equivalent** to computing the least squares solution!

Classification: Task Definition

Input

- A dataset $\mathcal{S} = \{(x_0, y_0), (x_1, y_1), \dots, (x_N, y_N)\}$ with $x_i \in \mathbb{X}^K$ and $y_i \in \mathbb{C}$
- $\mathbb{C} = \{C_1, \dots, C_L\}$ is **discrete** space of classes $C_1, \dots, C_L$
- We will only consider problems where each x_i can be assigned to exactly to class C_l ; the classes $C_1 \dots C_L$ therefore divide the input space $\mathbb{X}^K$ into **decision regions**
- Classification tasks with two classes $\mathbb{C} = \{C_1, C_2\} = \{0,1\}$ are also referred to as **binary classification**
- In general K classes are represented using a **1-of- K coding scheme** where $C_l = (0,0, \dots, 1, \dots, 0) \in \{0,1\}^K$

Desired Output

A **classifier** $h: \mathbb{X}^K \rightarrow \mathbb{C}$ that predicts y_n for a given x_n with minimum error

Generalized Linear Models for Classification

The output of linear regression is a weight vector w that defines a linear function of the form

$$y_{reg}(x_n, w) = \sum_{j=0}^M w_j \phi_j(x_n)$$

$y(x_n, w)$ maps input values to a continuous space $\mathbb{Y}^L$. Applying this type of model to classification requires an additional **nonlinear activation function** f that discretizes the output space:

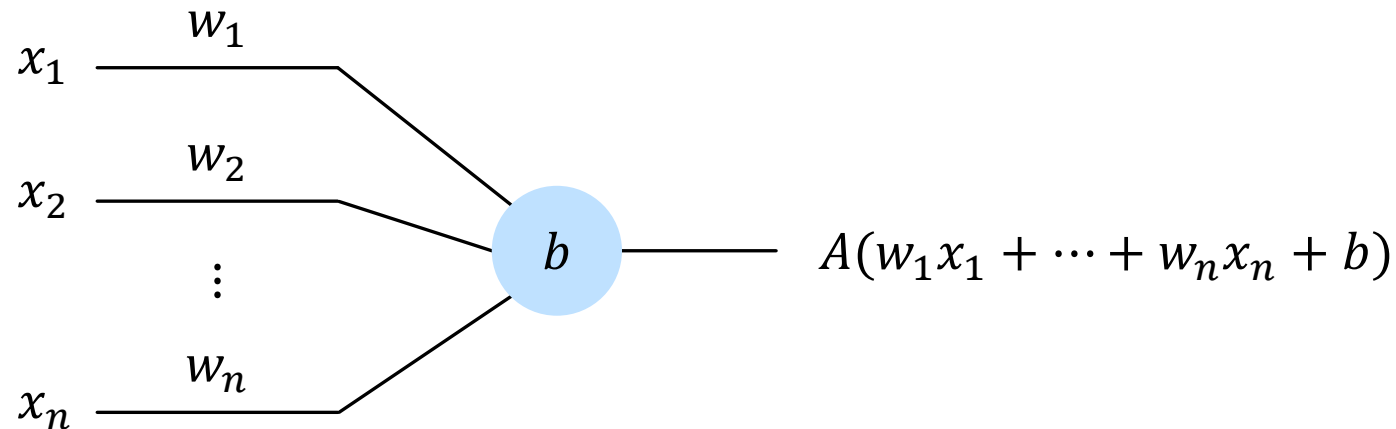
$$y_{cls}(x_n, w) = f\left(\sum_{j=0}^M w_j \phi_j(x_n)\right)$$

y_{cls} is called a **generalized linear model**. Subspaces $X^* \subseteq \mathbb{X}^K$ with $y_{cls}(x_n, w) = c$ for a constant $c \in \mathbb{C}$ are called **decision surfaces**. The decision surfaces are linear functions of x :

$$y_{cls}(x_n, w) = c \leftrightarrow \sum_{j=0}^M w_j \phi_j(x_n) = c^*$$

Recap: Artificial Neurons

An **analog artificial neuron** is defined by its **synaptic weights** and an **activation function**:



The output $n(x)$ of a neuron n with activation function A , synaptic weights w , bias b , and input x is computed as follows:

$$n(x) = A(w_1x_1 + \dots + w_nx_n + b) = A(w^T x + b)$$

→ **Analog neuron models are generalized linear models!**

Linear Separation for Binary Classification

Basic Idea

Two classes C_1 and C_2 can be linearly separated based on a linear **discriminant function**

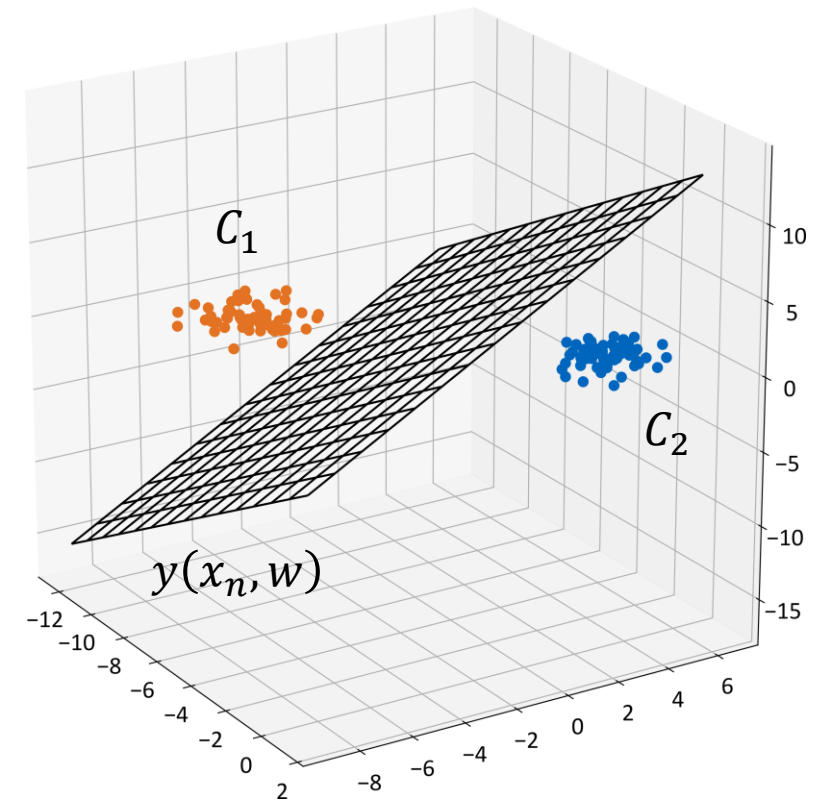
$$y(x_n, w) = w^T x_n = c^*$$

Mapping to C_l

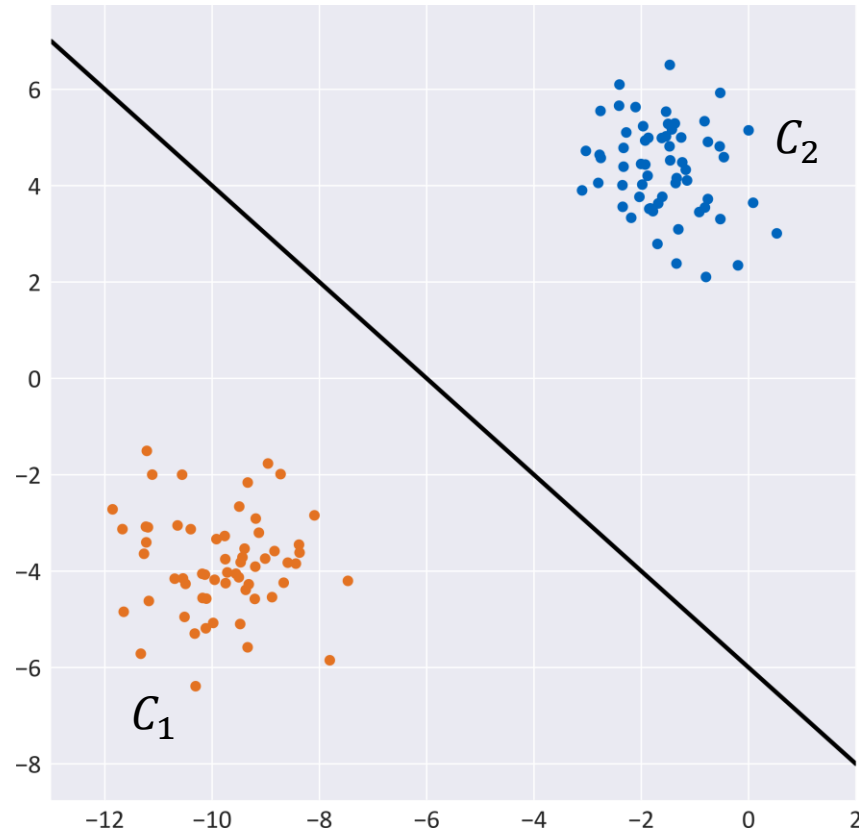
The class of a data point x_n is determined based on the value of $y(x_n, w)$. The most common activation function is the **Heaviside step function** f_H :

$$f_H(x) = \begin{cases} 0 & \text{if } n < 0 \\ 1 & \text{if } n \geq 0 \end{cases}$$

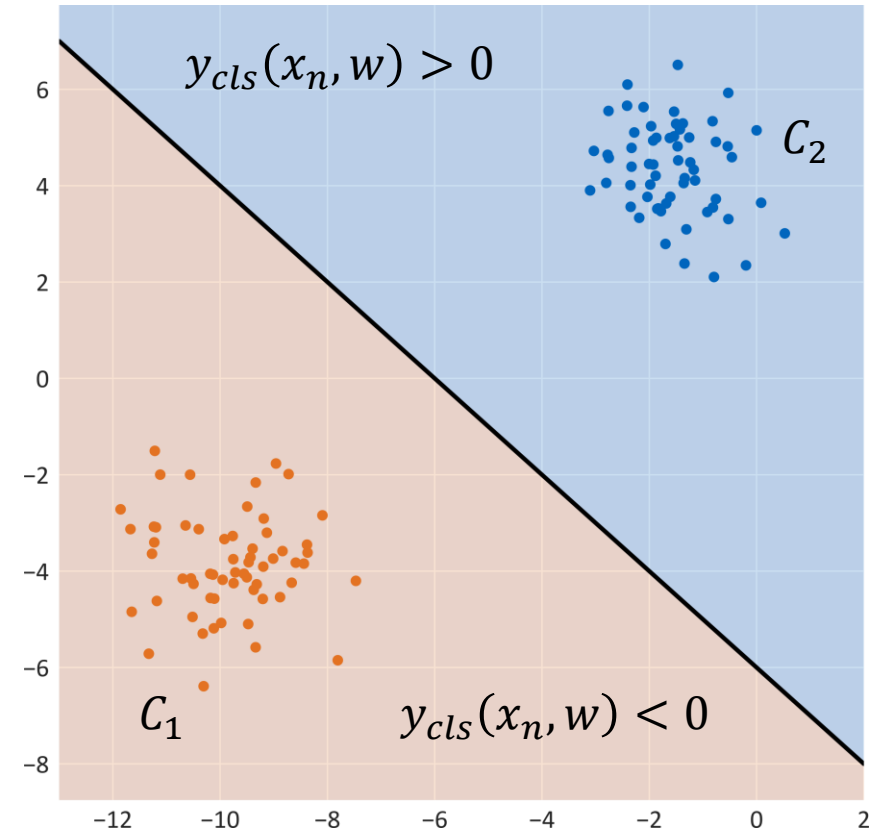
The resulting decision surface is $w^T x_n = 0$



Linear Separation for Binary Classification



The two classes C_1 and C_2 be separated by the line
 $w_0 + w_1x_1 + w_2x_2 = 0$



The decision surface divides the definition space of the samples into two half-spaces

Classifying Multiple Classes

In general, classifiers for more than two classes can be constructed by combining discriminant functions that classify subsets of $\mathbb{C} = \{C_1, \dots, C_K\}$. However, the two most evident approaches suffer from a serious issue:

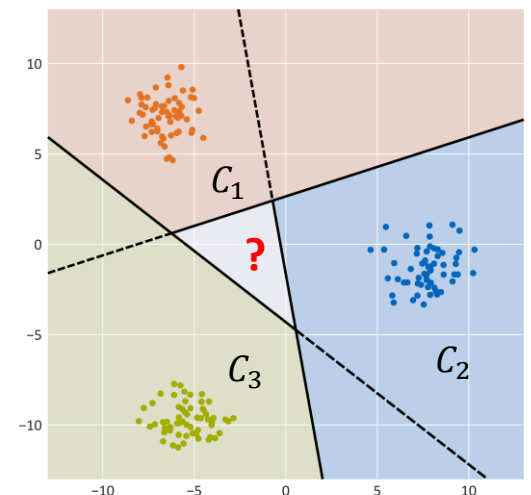
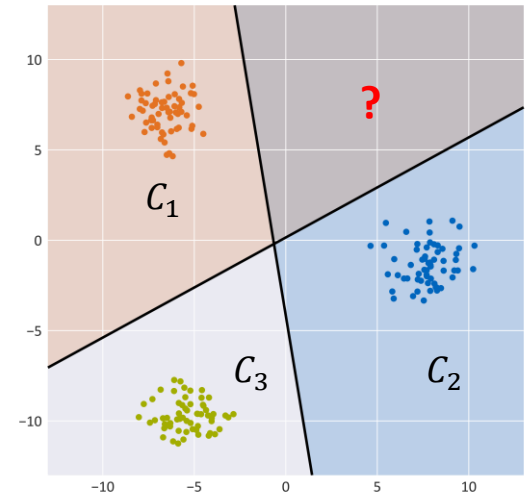
One-Versus-the-Rest Classifier (Top)

- Separation of K classes with $K - 1$ binary discriminant functions
- Every discriminant function separates one class from all others

One-Versus-One-Classifier (Bottom)

- Pairwise separation of K classes with $K(K - 1)/2$ binary discriminant functions
- The class of a data sample is assigned by a majority vote

In both cases, some regions are classified ambiguously!



Classifying Multiple Classes

Problem Analysis

Both one-versus-the-rest and one-versus-one classification fail because the input space regions defined by the discriminant functions **overlap** or do **not cover** the whole input space.

→ Determine a **partition** of the input space

Solution

Every class C_k is assigned a linear function of the form

$$y_{C_k}(x_n, w_k) = w_k^T x_n$$

A sample x_n is assigned to class C_k if the $y_{C_k}(x_n, w)$ of that class is larger than for all other classes:

$$C^* = \operatorname{argmax}_{C_k \in \mathbb{C}} y_{C_k}(x_n, w_k)$$

Classifying Multiple Classes

The decision regions defined by the classifier

$$C^* = \operatorname{argmax}_{C_k \in \mathbb{C}} y_{C_k}(x_n, w)$$

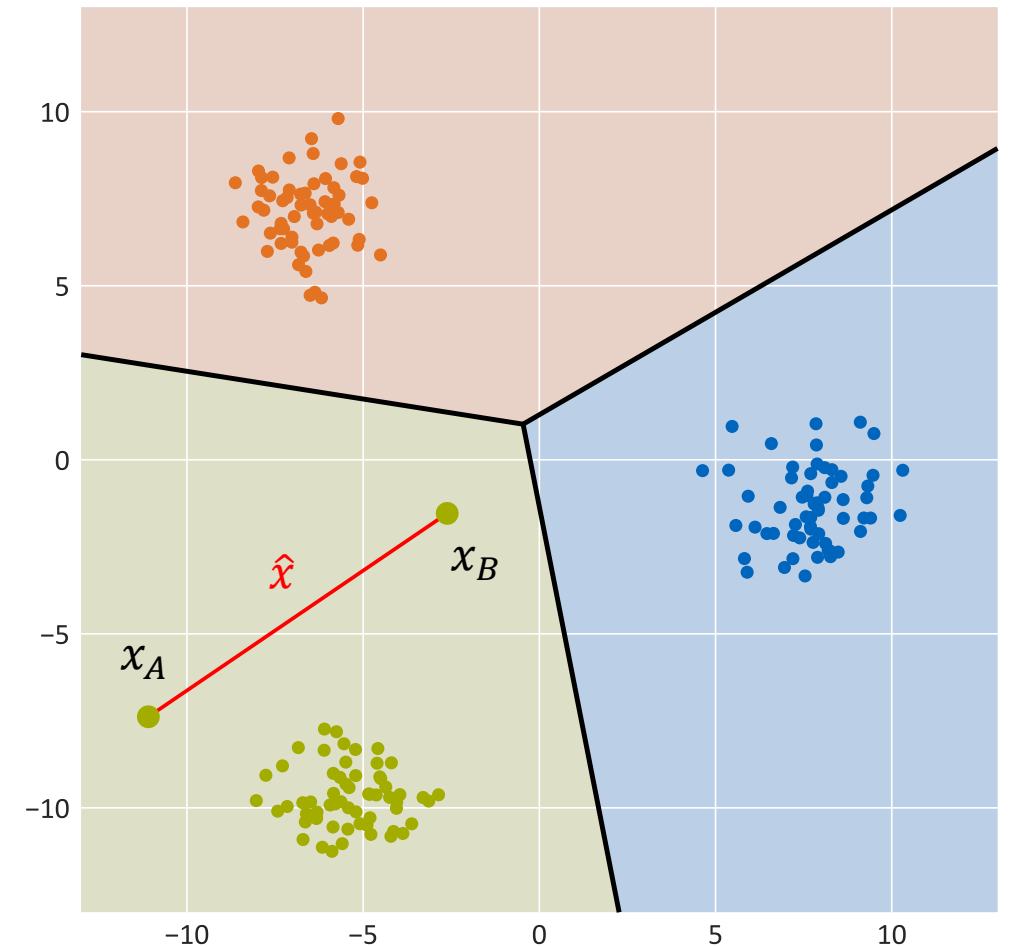
are always **convex**. Consider two points x_A and x_B in a decision region C_i . Any point $\hat{x}$ on the line connecting them can be expressed as

$$\hat{x} = \lambda x_A + (1 - \lambda)x_B \text{ with } 0 \leq \lambda \leq 1$$

Since the functions y_{C_k} are linear, it follows that

$$y_{C_k}(\hat{x}) = \lambda y_{C_k}(x_A) + (1 - \lambda)y_{C_k}(x_B)$$

With both x_A and x_B lying in the same decision region, it finally follows that $y_{C_i}(\hat{x}) > y_{C_k}(\hat{x})$ for all $i \neq k$, i.e. $C_i = \operatorname{argmax}_{C_k \in \mathbb{C}} y_{C_k}(x_n, w)$.



Representing Classes

A Naïve Approach

The classes C_k are elements of a **discrete** set $\mathbb{C}$. The choice of $\mathbb{C}$ can have a major impact on the learning process. The probably most obvious choice is to set $\mathbb{C} = \{1, 2, 3, \dots, L\}$:

$$C_1 = 1$$

$$C_2 = 3$$

$$\vdots$$

$$C_L = L$$

Problem

Assigning class identifiers from an **ordered set** implicitly also imposes a **not existing** order on the classes that might be captured by the learned model.

→ **Use one-hot encoding!**

One-Hot Encoding

To avoid introduction any latent information contained in ordered sets, a common scheme of encoding clusters and classes is **one-hot encoding**:

- For a data set with L clusters/classes set $\mathbb{C} = \mathbb{R}^L$
- Cluster/class C_k is assigned the vector $c_k = (0, \dots, 0, 1, 0, \dots, 0)^T$ where every except for the k th entry is 0

Example

Sample	House	Car	Cat	Flower
s_1	0	0	1	0
s_2	0	1	0	0
s_3	0	0	0	1
s_4	1	0	0	0
s_5	0	1	0	0

Least Squares for Classification

Encoding categorical class labels as one-hot vectors in $\mathbb{R}^n$ enables mapping the classification problem to a **regression problem** that can be solved with **least squares**. Recall that every class is described by its own linear model:

$$y_{C_k}(x_n, w_k) = w_k^T x_n$$

Grouping all functions y_{C_k} together yields:

$$y(x_n, W) = W^T x_n$$

W is a matrix that is comprised of the weight vectors w_k of the individual linear models y_{C_k} :

$$W = (w_0 \quad \dots \quad w_L)$$

Least Squares for Classification

Loss Function

With $X = (x_0^T, \dots, x_n^T)^T$ and $Y = (y_0^T, \dots, y_n^T)^T$, the **loss function** $\mathcal{L}_S$ of the least squares regression problem can be defined as follows:

$$\mathcal{L}_S(W) = \frac{1}{2} \text{Tr}\{(XW - Y)^T (XW - Y)\}$$

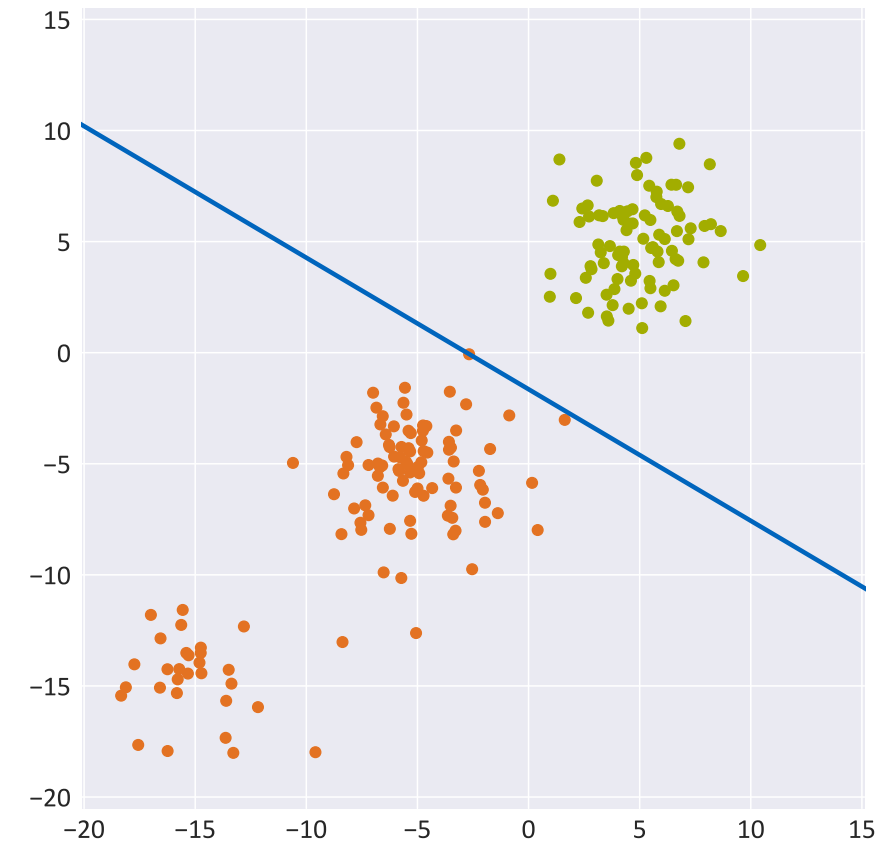
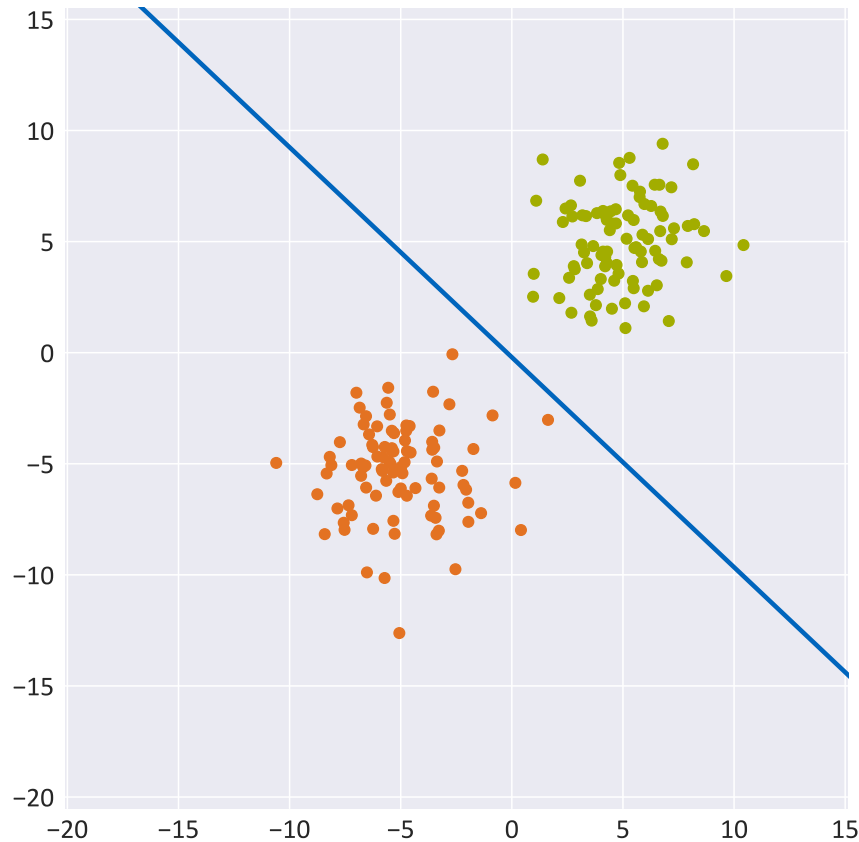
The **trace operator** ensures that only the quadratic terms on the diagonal of the product matrix account for the loss.

Least Squares Solution

Minimizing $\mathcal{L}_S(W)$ analogously to the standard regression case yields the weights values for the linear models y_{C_k} :

$$W = (X^T X)^{-1} X^T Y$$

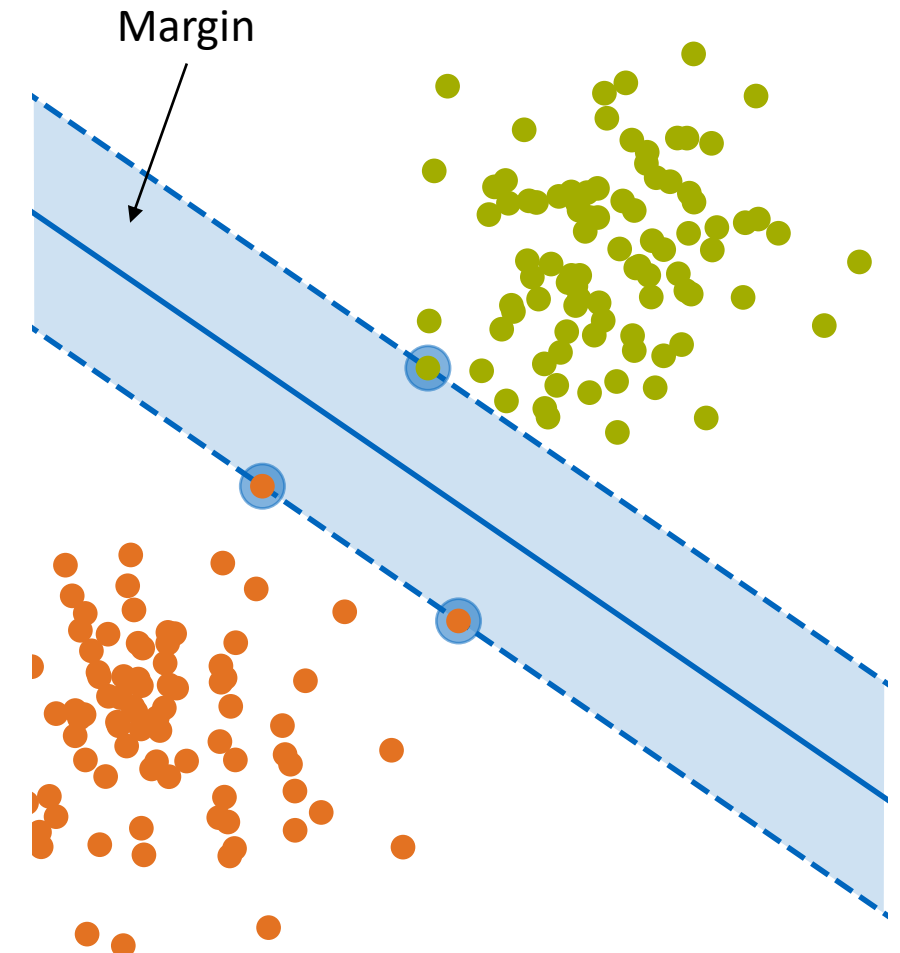
Issues of Least Squares Linear Classification



Least squares classification is sensitive to outliers!

Support Vector Machines

- When using least squares for linear classification, the resulting hyperplanes depend on **all samples** in the dataset. If the input is linearly separable, the algorithm computes one of **infinitely many solutions**.
- **Support Vector Machines (SVMs)** minimize the **generalization error** by computing a hyperplane that maximizes the **margin** of the classifier, i.e. the smallest distance between the decision boundary and the training samples.
- Maximizing the margin can be motivated using **computational learning theory**.



Support Vector Machines

Computing Distances from the Decision Boundary

Like in the standard classification task, the separating hyperplane of a SVM is defined as follows:

$$y(x) = w^T \phi(x) + b$$

By setting $\mathbb{C} = \{-1, 1\}$, all correctly classified examples fulfil the inequality

$$t_n y(x_n) > 0$$

The distance of a sample x_n from the decision boundary therefore can be computed as

$$d(x_n) = \frac{|y(x_n)|}{\|w\|} = \frac{t_n y(x_n)}{\|w\|} = \frac{t_n (w^T \phi(x) + b)}{\|w\|}$$

Support Vector Machines

Optimization Problem

Based on d , the optimization problem for computing the decision boundary can be formulated as follows:

$$\arg \max_{w,b} \left\{ \frac{1}{\|w\|} \min_n [t_n (w^T \phi(x) + b)] \right\}$$

By scaling w and b , one can easily set $t_n (w^T \phi(x^*) + b) = 1$ for the point x^* that lies closest to the decision surface. This results in the following optimization problem:

$$\arg \min_{w,b} \|w\|$$

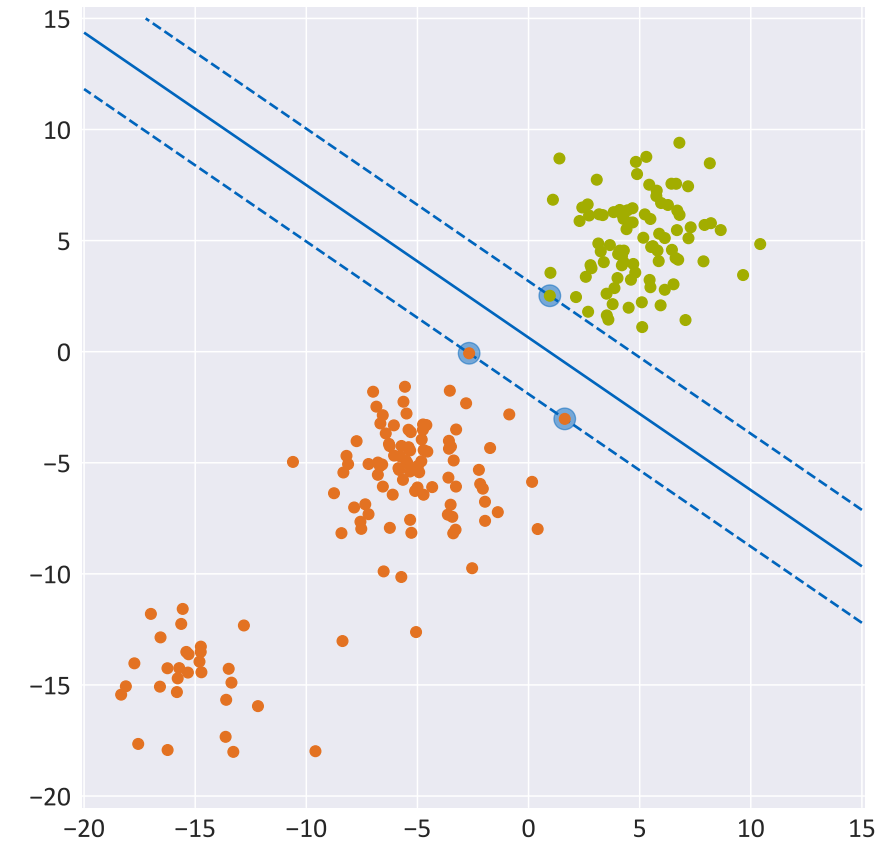
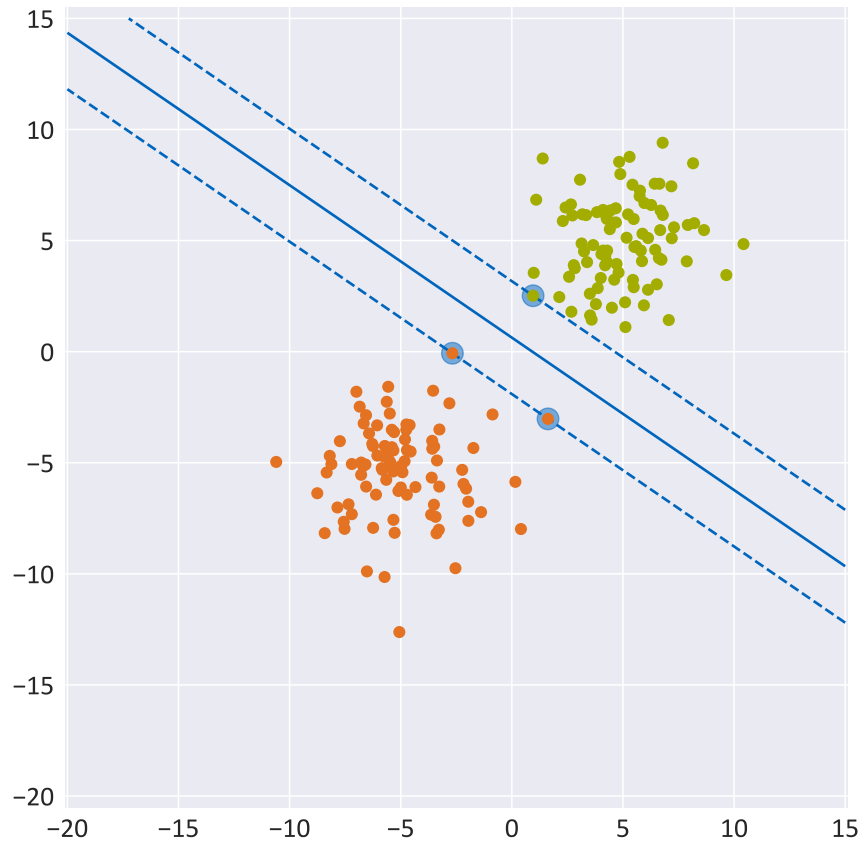
with $t_n (w^T \phi(x) + b) \geq 1$ for all samples x in the dataset

The above problem can be solved with quadratic programming (→ [Machine Learning lecture](#)).

Support Vector Machines

- Solving the optimization problem yields a set of **support vectors** that define the decision boundary of the SVM.
- The support vectors are samples from the **training dataset**.
- After training the SVM, all samples except for the support vectors can be discarded!
- In the general case, the optimization function also includes **slack variables** to allow for training samples that lie inside the margin. This ensures better **generalization performance** for overlapping classes.

Classification with SVMs



SVM classifier with support vectors and margins. Outliers do not affect the result.